Distributed Observability for Multi-Agent LLM Systems

A Shared-Log Approach on HPC Clusters

An observability framework for LLM-agent fleets on HPC clusters

---

Anna Monsó Rodriguez
A20653296

Advisor: Jaime Cernuda

Illinois Institute of Technology
Chicago, Illinois

August 11, 2026

Abstract

Applications built on large language models (LLMs) have outgrown the tooling built to observe them. They have changed in two ways at once: they have become multi-agent, and they have moved onto clusters. Observability tooling has followed one change or the other, never both: LLM-observability platforms capture rich agent semantics but have no notion of the physical machines involved, while HPC and fleet monitoring tracks node health at scale but carries no application semantics. To our knowledge, no existing tool spans the two.

This thesis begins from the observation that a multi-agent execution is already a causally-ordered event log, and tests the hypothesis that follows: if such a fleet's events are captured into a distributed shared log whose store places an append endpoint on every compute node, then agent semantics and cluster topology are coupled by construction, and a single reader over that log can answer the operator's fleet-scale questions.

To keep the design independent of any one store, we state the contract a store must honour: five required features (caller-named, ordered, durable, replayable, multi-reader streams) and four convenient ones whose absence costs a named mechanism rather than a redesign. Against that contract we design an observability framework for LLM-agent fleets, and implement it on ChronoLog, a distributed tiered log store that runs a capture process on every cluster node. Two capture paths feed the log (LLM turns through a node-local spool drained by a capture worker, inter-agent calls through per-node collectors), with at-least-once capture and effectively exactly-once ingest. Because that log is optimized for ingest and cannot be read inside its acceptance window, a hot/cold read design serves real-time views from the same event stream. On top of it we build fleet-scale analyses: failure detection that groups textually identical failures into ranked, diagnosed incidents at no token cost; per-node health rollups whose cost is independent of event volume; and cross-host critical-path tracing.

We validate the system end to end with real Claude agents on live 4- and 8-node ChronoLog clusters, passing 28 of 28 automated checks with up to 32 concurrent agents, and measure it throughout. Capture costs an agent 3.6 ms per turn, statistically invisible against real LLM latency. Events reach the operator's screen in 10.7 ms, against a 20–30 s durable drain. Fleet queries stay flat as event volume grows 50×. A ground-truth fault study scores 580 of 580 detections with zero false positives. In a timed case study, a first-time operator produced a complete, verified cross-layer diagnosis in 72 seconds, a task they could not begin from the raw logs. The implementation is a research prototype: it is validated at the scales reported here, and its limits are stated alongside the results.

Contents

List of Figures

List of Tables

Acronyms

LLM	Large Language Model
MCP	Model Context Protocol
HPC	High-Performance Computing
SLURM	Simple Linux Utility for Resource Management
SSE	Server-Sent Events
API	Application Programming Interface
SDK	Software Development Kit
GIL	Global Interpreter Lock
NFS	Network File System
JSONL	JSON Lines
CSV	Comma-Separated Values
HDF5	Hierarchical Data Format, v5
p95	95th percentile
RSS	Resident Set Size
APM	Application Performance Monitoring
eBPF	extended Berkeley Packet Filter
DNS	Domain Name System
UI	User Interface
RPC	Remote Procedure Call

Chapter 1

Introduction

This chapter motivates the thesis: it describes the workload (fleets of LLM agents on HPC clusters) and the observability gap it creates, states the observation the design builds on, and lists the contributions and the document structure.

1.1 Context: Multi-Agent LLM Systems on Clusters

Applications built on large language models (LLMs) have changed shape quickly over the last few years. The first generation consisted of single request–response calls: a prompt in, a completion out. The second generation wrapped the model in a loop, the *agent*, in which the model is prompted with a task and a transcript, emits a call to an external tool, has the tool's result appended to that transcript, and is prompted again, until it emits a final answer instead of a call [55]. Section 2.5.1 gives the mechanism precisely; the term *agent* is used throughout this thesis for that loop and the process running it, and for nothing more. The current generation goes one step further and decomposes a task across *multiple* cooperating agents, each with its own role, context, and conversation, which delegate sub-tasks to one another through structured tool-call protocols such as the Model Context Protocol (MCP) [3]. Multi-agent frameworks make this pattern routine [52]: a planner agent delegates sub-tasks to worker agents, workers consult specialist agents, and the overall computation is a conversation *between* programs as much as within them.

At the same time, the deployment platform of these systems is changing. When agents were single processes, a laptop or a single cloud instance was enough. Multi-agent workloads (scientific-workflow assistants, large-scale code analysis, data-processing fleets) instead run on *clusters*: tens to hundreds of nodes, allocated through batch schedulers such as SLURM [56], with several agents per node. Recent work places LLM-agent orchestration directly on leadership-class HPC systems [43] and on the workflow infrastructure that spans them [50]. The result is a genuinely distributed system in the classical sense: many processes on many machines, communicating over a network, with no shared clock and no single point of observation.

This combination is operationally demanding in a way neither ancestor was. LLM agents are non-deterministic, expensive (every turn has a token cost), slow (seconds per model call), and failure-prone in unfamiliar ways. They retry, emit calls naming tools or arguments that do not exist, hang on remote calls, and silently degrade; empirical studies of multi-agent LLM systems find such failures to be systematic rather than incidental [11], and the retry loops in particular are an instance of the amplification pathologies long known to destabilize distributed systems [10]. Running *fleets* of them across a cluster multiplies these problems by the number of nodes and adds the classical failure modes of distributed infrastructure: slow nodes, network partitions, and stragglers that drag down the whole computation [21]. An operator needs to answer questions that span both layers at once: *What is the fleet doing right now? Which agents are talking to which? Why is this scenario slow, and which hop is the bottleneck? Which node is dragging the fleet down? Has this failure happened before?*

This setting is the object of study of this thesis: a fleet of LLM agents distributed across the nodes of a high-performance-computing (HPC) cluster, and the observability infrastructure required to operate it.

1.2 The Gap: Agent Semantics vs. Cluster Topology

Observability tooling for this setting splits into two mature camps whose intersection is, to our knowledge, unoccupied. The first camp, *LLM and agent observability* (LangSmith [32], Langfuse [33], Arize Phoenix [5], AgentOps [2], the LLM products of established APM vendors [14], and the OpenTelemetry generative-AI semantic conventions now consolidating them [41]), captures rich agent semantics (prompts, tokens, costs, tool calls, rendered as traces of nested spans in the lineage of Dapper [49]) but treats the physical machines the agents run on as, at best, opaque metadata: host identity is not a dimension the tools can group, compare, or alert over. The second camp, *HPC and fleet monitoring* (Prometheus [45] with Grafana over batch-scheduled clusters, and HPC-native collectors such as LDMS [1]), is built for that physical layer (per-node metrics at scale, over hundreds of nodes) but carries no application semantics at all: that the process saturating a node is an LLM agent mid-conversation, retrying a failed delegation, is invisible. Section 2.6 surveys both camps in detail.

The consequence fits in one sentence, which this thesis takes as its motivating problem:

> *The tools we survey answer "what did my agent do?" with a trace of one run on one machine. To our knowledge, none answers "what is my agent **fleet** doing across the cluster, and which node is dragging it down?"*

Answering that question requires coupling agent semantics to physical cluster topology in a single system, the intersection both camps leave open.

1.3 The Log-Based Approach

The technical starting point is an observation about the shape of the data. Everything worth capturing about a multi-agent system is an *event*: an LLM turn (prompt, response, tokens, latency, cost), an agent-to-agent call (caller, callee, correlation, outcome), a retained-context operation (an item added to or evicted from the material an agent's next prompt is assembled from, §2.5.2), a recovery or backtrack. These events carry timestamps and causal relationships: a delegation *happens before* the turns it triggers, which happen before the reply edge. A multi-agent execution, in other words, *is* a causally-ordered event log in the sense of Lamport [31].

If the data is natively a distributed event log, the natural place to store it is a *distributed shared log* [7, 29], the abstraction industrialized by Kafka [30] and since carried into production control planes [6]. This thesis uses the term *substrate* for that storage system: the log store underneath the observability system, whichever one it is. Agents append locally on the node where they run; the log provides ordering, durability, and archival; and every observability view is a *reader* over the same substrate. Ordering and durability become properties of the storage layer rather than of the instrumentation, and cross-agent causal reconstruction comes with the substrate rather than being bolted on.

This yields the hypothesis the thesis tests: *if the events of a multi-agent LLM fleet are captured into a distributed shared log whose store places an append endpoint on every compute node, then agent semantics and cluster topology are coupled by construction rather than by integration, and a reader over that one log can answer the operator's fleet-scale questions without further instrumentation.* It is testable in two parts. Section 3.3 makes the storage half checkable by stating the features a store must provide, so that any candidate can be scored against it; Chapter 6 measures the second half against the requirements of §3.2.

Which store provides the log is a separate question, and this thesis keeps it separate: §3.3 states the contract a substrate must honour (five required features, S1–S5, and four convenient ones, S6–S9, each of whose absence is priced in a named mechanism), and Chapter 4 is written against that contract. The implementation runs on ChronoLog [28], a distributed, tiered shared-log store that orders events by physical time and, crucially for this setting, places a log-capture process (a

keeper) on every cluster node, so that the mapping from an event to the machine that produced it is a fact about where the append landed rather than a field the instrumentation must be trusted to fill in. That property, S6 in the contract, is what no general-purpose commit log offers and what makes the coupling of agent semantics to cluster topology structural. Section 4.8 reports the ChronoLog-specific consequences in one place, so that the design and its instantiation can be read, and criticized, separately.

On this foundation the thesis designs, implements, and validates an observability framework for LLM-agent fleets on HPC clusters. It is a research prototype: complete enough to run real agent fleets on a live cluster and to be measured end to end, but validated at the scales reported in Chapter 6 rather than hardened for production use. Two capture paths feed the log, one for LLM turns and one for inter-agent calls, and a dashboard reconstructs every view (live agent and cluster topology, failure diagnosis, fleet health, cross-host tracing) purely from what was captured. Because a shared log of this class is optimized for ingest (archive reads trail appends by tens of seconds of drain latency, and on the deployed client a live replay can block a reader for minutes), the system adopts a two-path read design: the log is the durable *cold* path, and an in-process *hot* store serves real-time views from the same event stream at ingest time. That split is the price of one feature the substrate contract marks optional, and §4.7 says precisely what a store providing it would let a reimplementation delete. We validate the system end-to-end with real Claude agents on live multi-node ChronoLog deployments.

1.4 Contributions

This thesis makes five contributions:

1. **A substrate contract for log-backed observability.** We identify the minimal set of features a storage substrate must provide for an observability system of this shape to be implementable on it (S1–S5), together with a second tier (S6–S9) whose absence is survivable at a stated price, and we score representative shared logs against both (Chapter 3). The contract is what makes the rest of the design portable rather than particular: it separates the architecture from the workarounds one deployment required, and it lets a prospective adopter predict the cost of a port from a table rather than discover it during one.

2. **A two-path capture architecture over a distributed shared log.** LLM turns flow through a node-local *spool*, an append-only file on the agent's own machine, drained into the log by a separate *capture worker* process; inter-agent calls flow through a per-node *collector* daemon into that node's keeper. The design provides at-least-once capture and effectively exactly-once ingest, deduplicating on a per-session sequence number (a *high-water mark*), with crash recovery reconstructed from the log itself (Chapter 4).

3. **A hot/cold read design around the log's ingest-optimized trade-off.** The log store's drain latency (tens of seconds) and a replay path that can block the reader for minutes make it unreadable in real time; the system pairs it with an in-process *hot store* written at the moment of ingest, and a server-sent-events (SSE) *live bus* that pushes each event on to the browser, so that real-time views and durable full-fidelity views are projections of one event stream (Chapter 4).

4. **Fleet-scale analytics computed on the log.** Three analyses turn the captured events into operator answers: a failure-diagnosis pipeline, the *Doctor*, which detects failures, groups the textually identical ones into ranked incidents, and attaches zero-cost diagnoses with cross-run incident history; fleet-health *rollups*, per-(host, minute) buckets aggregated at write time, that make the cluster overview $O(\text{nodes} \times \text{buckets})$, independent of event volume; and cross-host critical-path tracing that names the bottleneck hop by *self-time*, its latency minus its slowest child's (Chapter 5).

5. **A measured evaluation on live clusters.** An automated end-to-end harness exercises the full stack with real Claude Agent SDK agents on live 4- and 8-node ChronoLog clusters, and we measure five dimensions: capture overhead, freshness, detection quality against planted ground truth, scalability in event volume and agent count, and a timed cross-layer diagnosis case study (Chapter 6). Section 6.8 bounds each of those claims: the scales actually validated, the diagnoser's coverage, and the substrate behaviour the design works around rather than fixes.

1.5 Document Structure

Chapter 2 develops the background (distributed-systems foundations, distributed tracing, the shared-log abstraction, ChronoLog, and LLM agents) and surveys the state of the art to establish the gap. Chapter 3 distills the target scenario into explicit requirements and states the substrate contract S1–S9. The system design follows in Chapter 4: the capture paths, the mapping of agent semantics onto the log, delivery semantics, and the hot/cold read split, written against that contract, with §4.8 collecting what the ChronoLog instantiation adds. Chapter 5 builds the fleet-scale analyses on top of it. Chapter 6 measures overhead, freshness, scalability, and end-to-end correctness, and closes with a fault-localization case study. Chapter 7 concludes.

Chapter 2

Background and State of the Art

This chapter develops the concepts the thesis is built from, as a progression: the distributed-systems fundamentals that define the problem's vocabulary (§2.1); distributed tracing, the discipline that first made cross-machine causality observable (§2.2); the distributed shared log, the storage abstraction this thesis adopts as its substrate (§2.3); ChronoLog, the concrete log store used (§2.4); LLM agents and multi-agent systems, the workload being observed (§2.5); and finally the state of the art, whose two disjoint camps define the gap this thesis targets (§2.6). Most sections close by noting what the thesis takes from them.

2.1 Distributed Systems Fundamentals

2.1.1 Time, Ordering, and Causality

A distributed system is a set of processes on separate machines that communicate only by exchanging messages. Its defining difficulty is that there is no global "now": each machine has its own clock, clocks drift, and no observer sees the whole system at once. Any statement about what the system *is doing* must therefore be reconstructed from locally recorded observations.

Lamport's insight is that such reconstruction turns not on wall-clock simultaneity but on *causality*, captured by the *happened-before* relation [31]: a precedes b if they occur in that order in one process, if a sends a message b receives, or transitively through a chain of such steps (Figure 2.1). It is a *partial* order, and it is the order that determines whether one event could have influenced another: an event later on the wall clock may be causally unrelated to an earlier one, and no amount of clock accuracy will connect them. Logical and vector clocks [31, 18, 36] make this order computable from local information; alternatively, when all events carry timestamps from reasonably synchronized physical clocks and land in one ordered store, causal chains follow from timestamps plus application-level correlation identifiers.

The corollary this thesis leans on is that the *event log*, an append-only sequence of timestamped, attributed events, is the canonical representation of distributed execution (Figure 2.1). A log captures exactly what each process observed and when; every richer structure (a request trace, a communication graph, a health summary) is a fold over it.

Used in this thesis: multi-agent executions are treated as partially ordered event logs; causal structure is recovered from physical-time ordering plus explicit correlation identifiers carried by the instrumentation (Chapters 4 and 5).

2.1.2 Delivery Semantics

Any pipeline that moves events from producers to a store must define what happens when a component fails mid-transfer. Three contracts are standard [27]. *At-most-once* delivery never retries, so failures lose events, which is unacceptable for an audit trail. *At-least-once* delivery retries until acknowledged, so failures duplicate events. *Exactly-once* delivery is the ideal, unachievable in general between separately failing components; real systems approximate it as *effectively exactly-once*: at-least-once transport paired with deduplication at the consumer, using idempotent writes or monotonic sequence numbers ("high-water marks") so that redelivered events are recognized and dropped.

A related design decision is where events wait when a downstream component is slow or dead. Buffering writes in a local, durable *spool* decouples the producer from downstream availability: the producer's fast path is a local append, and a separate drain process moves data onward at

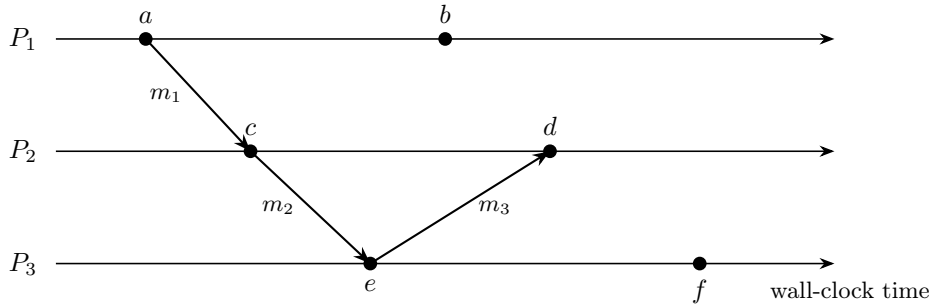


Figure 2.1: Happened-before as a partial order. Within a process, time runs left to right; the arrows m_1–m_3 are messages. The chain $a \to c \to e \to d$ is causal, and so is $a \to c \to e \to f$, because each step is either a step within one process or the delivery of a message. Event b is *concurrent* with c, d, e and f: no chain reaches it and none leaves it, so no ordering between them is defined, however their wall-clock times compare. This is the structure a multi-agent execution already has, with delegations as the messages (§2.5.1).

its own pace, the same write-ahead pattern that underlies journaling file systems and message brokers.

Used in this thesis: capture is at-least-once from a durable local spool; ingest into the log is made effectively exactly-once by per-session high-water marks, with consumer state rebuilt from the log itself after a crash (§4.4).

2.2 Distributed Tracing

2.2.1 Lineage: From Dapper to Pixie

Distributed tracing is the discipline of reconstructing, from per-process records, the cross-machine causal story of a single request. The idea predates its industrial form. Magpie [8] extracted per-request resource traces by stitching fine-grained kernel, middleware and application events into request paths, and X-Trace [19] propagated an explicit task identifier across layers and administrative domains so that a request's full path could be reconstructed rather than inferred. Google's Dapper [49] established the modern vocabulary: a *trace* is a tree of *spans*, each span a timed operation carrying the trace's identifier, its parent's identifier, and annotations. Two Dapper design tenets recur throughout this thesis. First, instrumentation must be low-overhead and effectively invisible to application developers, or it will be turned off or bypassed; Dapper achieved this by instrumenting shared infrastructure (the RPC library) rather than applications. Second, complete capture at scale is considered infeasible, so Dapper samples aggressively (uniformly, in its default scheme).

Subsequent systems relaxed different assumptions, and four moves matter here. The Mystery Machine [13] showed that when explicit propagation is unavailable, the causal model itself can be *inferred* from large numbers of logged executions, by hypothesizing happened-before relationships and pruning those contradicted by observed orderings. Pivot Tracing [34] made tracing *dynamic*, with queries installed at runtime that propagate context along the causal path. Canopy [26] scaled the pipeline at Facebook, emphasizing that raw traces are not what engineers consume: they are transformed into feature-indexed datasets backing purpose-built views. Pixie [44] pushed capture into the platform via eBPF, trading application-level semantics for zero-touch deployment.

2.2.2 Critical-Path Analysis and Self-Time

A trace answers "what happened"; the follow-up question is "why was it slow." In a tree of overlapping timed operations, total latency is not the sum of span durations: children execute inside their parents, and siblings overlap. The *critical path*, an idea imported into parallel-program performance analysis by Yang and Miller [54], is the chain of operations from root to leaf whose

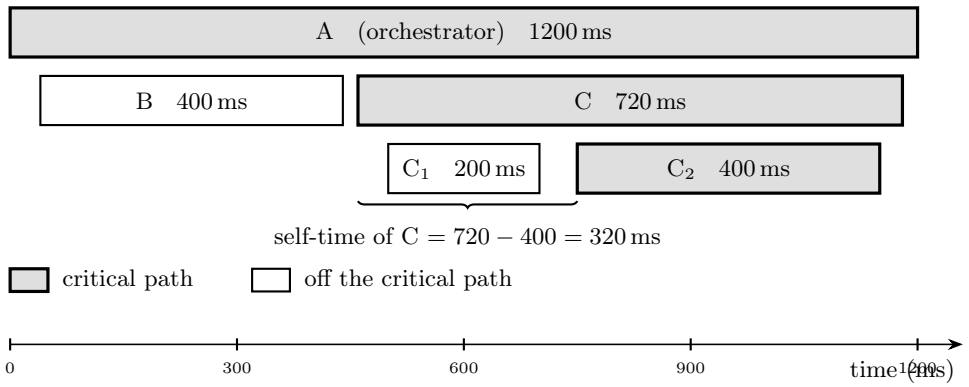


Figure 2.2: A trace rendered as a waterfall. Grey spans form the critical path $A \to C \to C_2$, the chain that determines end-to-end latency; white spans (B, C_1) overlap it and cannot shorten the request. Each hop is charged its *self-time*: C's 720 ms minus the 400 ms covered by its slowest child leaves 320 ms of C's own.

durations determine end-to-end latency: shortening any operation off the critical path cannot speed up the request. Within the critical path, the correct measure of each hop's contribution is its *self-time* (exclusive time): wall-clock time spent in the operation itself, excluding intervals covered by its children. Attributing latency by inclusive duration double-counts every level of the tree; self-time attribution names the hop actually responsible. Figure 2.2 illustrates both ideas on a small trace: shortening B or C_1 cannot speed up the request, and although C's inclusive duration is 720 ms, only 320 ms of it is C's own. The Mystery Machine used this analysis to find optimization opportunities at Facebook [13].

Used in this thesis: the tracing view of Chapter 5 reconstructs cross-host call trees from inter-agent events (using explicit parent correlation when present and Mystery-Machine-style inference from session and time nesting when not), computes the critical path, and names the bottleneck hop by self-time.

2.2.3 Sampling and Its Limits at Fleet Scale

Dapper's sampling assumption does not transfer cleanly to the LLM-agent setting. Head-based sampling (deciding at trace start) misses rare failures by construction; tail-based sampling (deciding at trace end) must buffer every trace until its fate is known, which at fleet scale reintroduces the data-volume problem sampling was meant to solve. LLM-agent workloads change the calculus in two ways. First, event rates are modest: an agent turn takes seconds and costs real money, so a 100-agent fleet produces tens of events per second where RPC tracing faces rates many orders of magnitude higher, which is why Dapper samples at all; complete capture is affordable. Second, the value of completeness is higher: each event is semantically rich, the failures are exactly the rare events sampling discards, and cost accounting requires totals, not estimates.

Used in this thesis: the system captures completely and treats storage, not sampling, as the scaling lever, pushing the volume problem down to the log store and the read-side rollups (§2.3, Chapter 5).

2.3 The Distributed Shared Log Abstraction

The systems of §2.2 all separate capture from storage: spans are generated in-process, then shipped to a backend whose design is incidental to the tracing model. A complementary line of work treats the *log itself* as the system's central abstraction. Kreps's formulation [29] argues that an append-only, totally ordered log is the natural integration point for distributed data systems: producers append, consumers read at their own pace, and the log's order *is* the system's notion of time. Kafka [30] industrialized this as partitioned commit logs (replication followed in later

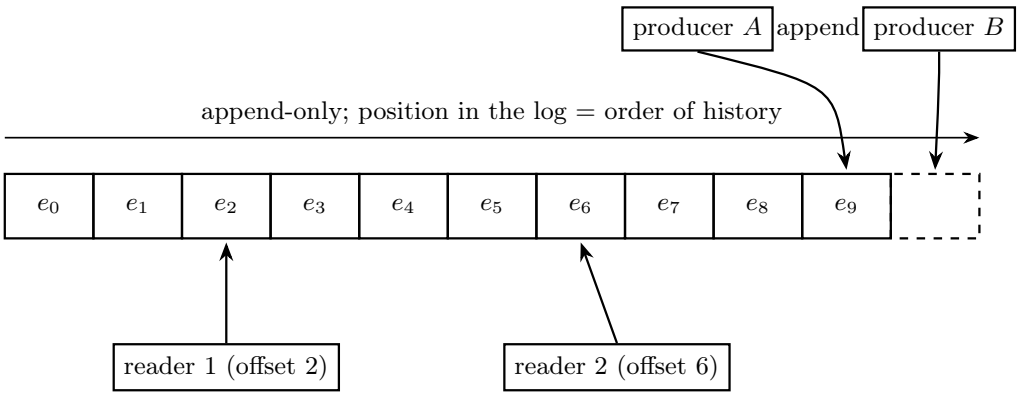


Figure 2.3: The shared-log abstraction. Producers append at the tail and readers consume at independently held offsets, so a reader can be slow, restart, or replay from the beginning without any producer noticing. Position in the log *is* the order of history, which is what lets every view in this thesis be a fold over one sequence rather than a separately maintained store.

versions); CORFU [7] showed that a shared log can be a first-class distributed primitive: clients append to a global address space striped over a cluster, and the shared log's total order becomes the foundation for replicated state machines and transactional systems built above it.

Three properties matter for observability (Figure 2.3): *node-local appends* keep the producer's fast path off the network; *decoupled readers* impose no back-pressure, so an expensive analysis and a cheap dashboard read the same log without perturbing capture; and *durability with order* makes an event's place in history permanent, which is what an audit trail of agent behavior needs. Section 3.3 restates them as the contract the design actually depends on.

The abstraction also has a characteristic trade-off: log stores are *write-optimized*. Appends are fast because data lands in ingest-side buffers that are sealed, ordered, and moved to read-optimized storage asynchronously, so a reader wanting the most recent events must either wait out that pipeline or read a separate, freshness-oriented structure. Stream-processing architecture formalized the resulting design as the *lambda architecture*: a durable, complete, high-latency batch path beside a fast, approximate speed layer over the same event stream [35]. This trade-off drives a central design decision of this thesis (§2.4.3, Chapter 4).

Used in this thesis: the shared log is the single substrate (capture appends to it, every view reads from it), and its write-optimized/read-lagging nature is handled by a lambda-style hot/cold split rather than by abandoning the abstraction.

2.4 ChronoLog

ChronoLog [28] is a distributed, tiered shared-log store developed for HPC environments, and the concrete substrate of this thesis. The paper presents the design; the component names, tier formats, and configuration described below reflect the deployed version used in this work. It differs from the logs of §2.3 in one decision that makes it particularly suited to this thesis's workload: events are ordered by *physical time* at capture, rather than by a sequencer-assigned logical position, so the log's global order directly reflects when things happened across the cluster.

2.4.1 Architecture: Visor, Keepers, Grapher–Player

A ChronoLog deployment (Figure 2.4) comprises a coordinating *visor*, a recording group of *keepers*, and an archival pipeline of *grapher* and *player* processes.

The visor is the control plane: clients connect to it to create and acquire logs and to discover the recording group. The keepers are the ingest plane: one keeper runs on every node of the deployment, and clients append events to a node-local keeper, which buffers recent history in memory. Asynchronously, the grapher drains sealed chunks from the keepers, merges them into

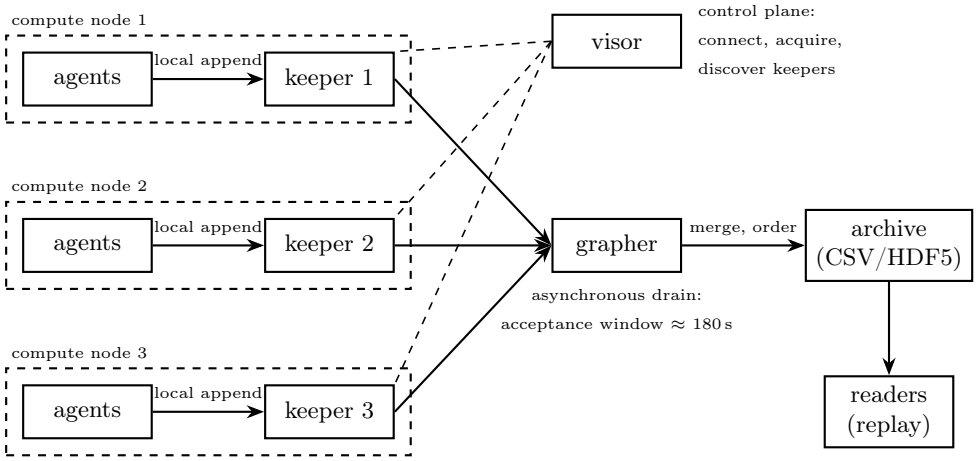


Figure 2.4: ChronoLog's architecture. A keeper on every compute node takes node-local appends into memory (the ingest tier); the visor coordinates clients; the grapher asynchronously drains, merges, and time-orders sealed chunks into a read-optimized archive. The drain imposes an acceptance window during which fresh events are not yet readable (20–30 s measured end-to-end here, with a 180 s window configured at the grapher). This is the property the read path of Chapter 4 is designed around.

time order, and writes them to a read-optimized archive (CSV/HDF5 tiers), which the player serves back to readers. The design mirrors the tiered storage hierarchies of HPC I/O systems: a fast, distributed, memory-resident ingest tier in front of a durable, ordered archival tier.

Two consequences of this architecture matter for this thesis. The keeper-per-node layout means capture is *physically co-located* with the workload: an agent's events are appended on the machine where the agent runs, and the mapping from events to nodes (the coupling of semantics to topology that Chapter 1 identified as the gap) is native to the substrate rather than reconstructed. And the drain pipeline means the archive is complete and ordered but *late* (§2.4.3).

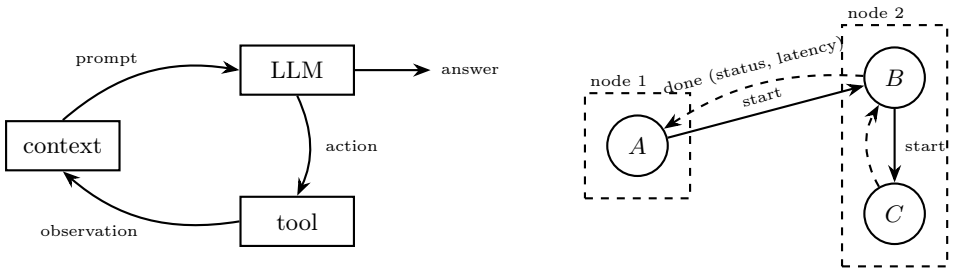
2.4.2 Data Model: Chronicles, Stories, Events

ChronoLog organizes data hierarchically. A *chronicle* is a named log namespace; within it, a *story* is an independently written and read event sequence; an *event* is a timestamped payload appended to a story. Clients acquire a story handle to append or replay, and stories provide the unit of isolation: writers to different stories do not contend, and a reader replays exactly one story's history in time order. The model maps naturally onto observability data: chronicles partition by data kind, stories by session or entity, and events carry the individual observations (§4.3 details this thesis's schema).

2.4.3 Design Point: Ingest-Optimized, Delayed Reads

ChronoLog's design point is high-throughput, low-latency *ingest*: appends land in the node-local keeper's memory and return. The price is paid on the read side. The keeper→grapher drain imposes an *acceptance window* during which freshly appended events are held in ingest buffers and are not yet readable from the archive: in the deployments used here the keeper seals 10 s chunks under a 15 s acceptance window, for a measured end-to-end availability of 20–30 s (§6.3), while the grapher and player run with a longer, 180 s window.

This is not a defect but the standard write-optimized trade-off of §2.3 in concrete form, and it dictates the read-side architecture of Chapter 4: ChronoLog serves as the durable, complete cold path, while real-time views are fed by a hot path populated at ingest. That is the lambda split, with the acceptance window as the boundary between the layers. Two further properties of the deployed system, the blocking behavior of a read that lands inside the window and the absence of a catalog operation, shape the design just as strongly; §4.2 states all three together, where the responses to them belong.



(a) the agent loop: one *turn* per LLM call (b) delegation: agents as tools, calls across nodes

Figure 2.5: (a) The agent loop: each cycle through the LLM is one *turn*, appending the tool's observation to the agent's context. (b) Multi-agent delegation: once agents are tools, agent A calls agent B (possibly on another node), which calls C in turn, each call a `start` with a matching `done` (dashed) carrying status and latency. The result is a partially ordered event log with delegations as its cross-process edges.

2.5 LLM Agents and Multi-Agent Systems

2.5.1 The Agent Loop, Tool Use, and Delegation

An LLM *agent* couples a language model to an environment through a loop (Figure 2.5a): the model is prompted with a goal and a transcript of prior steps; it emits either a final answer or an *action*, a call to an external tool; the tool's result is appended to the transcript; and the loop repeats, a pattern crystallized by ReAct [55]. The unit of execution is the *turn*: one model invocation, with its prompt, response, latency, and token consumption. It is the unit this thesis captures.

Tool interfaces are standardizing around protocols such as the Model Context Protocol [3], in which tools are typed, discoverable endpoints. Once tools are first-class, *agents themselves can be tools*: one agent's toolset includes a call-remote-agent operation that opens a sub-conversation with another agent, possibly on another machine. This is how multi-agent systems form [52]: a scenario is a set of agents, each a loop over its own context, connected by delegation edges. From a distributed-systems standpoint (§2.1.1), a scenario execution is a set of per-agent event sequences joined by inter-agent messages: precisely a partially ordered event log, with delegation calls as the cross-process edges.

2.5.2 Observable Signals

The observable surface of an agent fleet has two planes. The *intra-agent* plane is the sequence of LLM turns: prompt and response content, model identity, latency, token counts, and (unusual among workloads) direct monetary cost per event, since providers bill per token. It also includes the agent's *retained context*: the set of messages, tool results, and retrieved material assembled into each turn's prompt, which grows as results accumulate and shrinks under eviction and summarization when the model's context window fills. A terminological note, since the alternative is common enough to cause confusion: the agent literature often calls this the agent's *working memory*, by analogy with the cognitive-psychology construct. This thesis avoids that name and uses *retained context* throughout, because the analogy imports connotations the mechanism does not have. Nothing is remembered; the retained context is simply the material the next prompt will be assembled from, and an entry leaves it when an eviction policy drops it, not when anything is forgotten. Its dynamics are nonetheless an operational signal in their own right: an agent nearing its context limit degrades before it fails, and the additions and evictions are therefore captured as events (§4.3). The *inter-agent* plane is the set of delegation calls: which agent called which, when the call started and finished, whether it succeeded, and how calls nest. This is the plane from which scenario structure, communication topology, and cross-host traces are reconstructed.

2.5.3 Failure Modes of LLM Agent Fleets

Agent fleets fail in ways that combine both planes. Empirical taxonomies of multi-agent LLM failures [11] find the dominant categories to be specification gaps, inter-agent misalignment, and missing task verification, all of which surface to an operator as the concrete symptoms below. On the intra-agent plane: provider errors (HTTP 4xx/5xx, rate limits), latency outliers against a model's typical distribution, and *retry storms*, in which an agent re-issues a failing prompt in a loop, a classic amplification pathology (the retry-driven feedback loops of metastable failure [10]), here with a direct monetary cost attached to every retry. Agents may also *backtrack*, restoring earlier context after an unproductive path; frequent recoveries signal a struggling agent. On the inter-agent plane: failed delegations, and, particularly damaging, *hung* calls (termed *orphan calls* from Chapter 5 on), in which a delegation starts but never completes, silently stalling the delegating agent; in a per-edge view nothing is visibly wrong, since the signature is an absence. At fleet scale these interact with infrastructure pathology: a degraded, fail-slow node [21] drags down every agent scheduled on it, producing a symptom that is scattered across scenarios yet has a single physical cause. Localizing such failures is the cross-layer question neither tooling camp of §2.6 can answer alone.

2.6 State of the Art and the Gap

2.6.1 LLM and Agent Observability

The first camp instruments the application layer. LangSmith [32] (from the LangChain ecosystem), Langfuse [33], and Arize Phoenix [5] capture LLM calls and tool invocations as hierarchical runs or OpenTelemetry-style spans, adding LLM-specific value: token and cost accounting, prompt management, dataset curation, and LLM-as-judge evaluation. AgentOps [2] targets agent frameworks specifically, tracking sessions, tool usage, and multi-agent hand-offs. General application-performance-monitoring (APM) vendors have added LLM-observability products [14], and the OpenTelemetry project is standardizing generative-AI semantic conventions [41], signalling consolidation of this camp around the span model.

The limitation is structural, not incidental. The span model these tools inherit was designed for request-scoped microservice tracing: "distributed" means span correlation across services in one application's request path; "multi-agent" means multiple agent abstractions within one process or one orchestrator. Host identity, where it appears at all, is opaque metadata: a tag an operator can attach and read back, not a first-class dimension the tools can group by, compare across, or alert over. Population-level questions (compare nodes, find the sick one, correlate agent failures with machine placement) therefore cannot be phrased. The nearest exceptions prove the shape of the gap rather than filling it: an APM platform such as Datadog can correlate an LLM trace with the host metrics of *that request's* infrastructure, but offers no fleet-of-agents semantics over an HPC allocation: no batch-scheduler awareness, no population view keyed by agent behavior. The OTel generative-AI conventions standardize *turn* semantics, while the host resource attributes an SDK may attach come with no defined fleet-level analysis. In both cases the data model cannot express the join the operator questions of §1.1 require: from a fleet-wide symptom to a specific agent's conversation on a named machine.

2.6.2 HPC and Fleet Monitoring

The second camp instruments the physical layer, and does so at scale. Prometheus [45], typically paired with Grafana dashboards, is a de facto standard for metrics-based monitoring, routinely deployed over SLURM clusters via node exporters and job accounting; HPC-native systems such as LDMS [1] are engineered for continuous, low-overhead collection of node metrics across very large machines. These systems answer per-node and per-job questions (utilization, saturation,

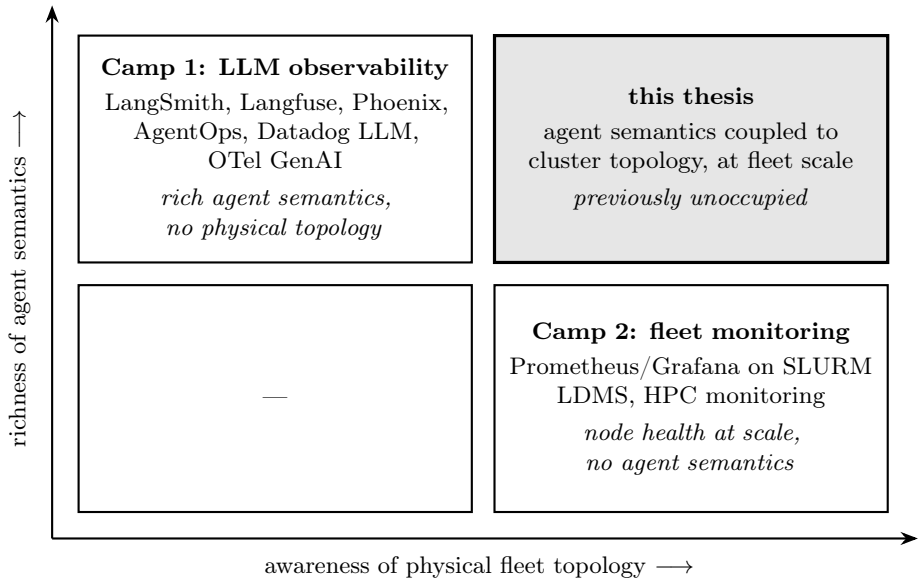


Figure 2.6: The observability landscape for multi-agent LLM systems on clusters, along its two capability axes. Camp 1 and Camp 2 each occupy one off-diagonal quadrant; their intersection (agent semantics coupled to physical fleet topology) is unoccupied among the surveyed tools, and is the quadrant this thesis targets.

hardware health, stragglers) well, and their visual idioms (heatmaps, per-node grids) are built for hundreds of nodes.

Their blindness is the mirror image of the first camp's. The data model is metric time series named by host and counter; there is no application semantics at all. That a CPU-saturated node is running an LLM agent whose retry storm is the actual pathology (and that the same storm is visible as cost and latency in some scenario's transcript) is invisible: metrics cannot say *which conversation* is responsible, and offer no path from a red node to the agent behavior causing it.

2.6.3 The Empty Intersection

Figure 2.6 summarizes the landscape along its two axes: richness of agent semantics, and awareness of physical fleet topology. Camp 1 occupies the semantics-rich, topology-blind quadrant; Camp 2 the topology-aware, semantics-blind one. The quadrant combining both (agent semantics coupled to cluster topology, at fleet scale) is, among the tools surveyed here, unoccupied. It is where the operator questions of §1.1 live: they require pivoting *between* layers, from a fleet-level symptom to a single agent's conversation and back.

Filling that quadrant is not a matter of running both camps side by side: two disjoint systems with disjoint data models provide no join between an agent event and a node, which is precisely the pivot the questions require. The evaluation returns this structural argument as a measurement (§6.7): faced with an injected cross-layer fault on a live cluster, an operator given the raw captured files (all the signals both camps would collect, join included) could not produce a single localization answer. Through the joined views, that operator completed the full symptom-to-conversation pivot in 72 seconds. The same experiment also separates the gap's two halves: sufficiency is a property of the *data model* (an expert reader extracted every answer from the log files alone, because host attribution and causal correlation are in the events themselves; §6.7 gives the details), while accessibility is a property of the *interface* built on that model. The camps fail at the first half, which no interface can repair: a view over data that lacks the join cannot invent it. The gap therefore calls for a single substrate in which agent-semantic events are natively attributed to physical nodes: what the shared-log approach of §2.3, on ChronoLog's keeper-per-node architecture (§2.4), provides by construction. Turning that substrate into a working observability system is the subject of the rest of this thesis.

Chapter 3

Problem Statement and Requirements

This chapter turns the gap established in Chapter 2 into a concrete engineering problem: a target scenario (§3.1), a set of requirements that any solution must satisfy (§3.2), the contract the storage substrate must honour for such a solution to be implementable at all (§3.3), and the boundaries of what this thesis does not attempt (§3.4). The requirements are numbered R1–R7 and are used as the evaluation rubric in Chapters 6 and 7; the substrate features are numbered S1–S9 and are the interface against which Chapter 4 is written.

3.1 Target Scenario

The setting is a multi-agent LLM workload on an HPC cluster. A user holds a SLURM [56] allocation of M compute nodes (in this thesis's experiments, $M \in \{4, 8\}$ on the ARES cluster, with analyses exercised up to several hundred simulated nodes). On each node run K LLM agents: independent processes, each an agent loop over its own conversation context, each identified by a scenario-scoped session identifier of the form `role@scenario`. Agents cooperate: an agent may delegate a sub-task to a peer on another node by invoking a remote-agent MCP [3] tool, producing nested chains of cross-host calls. A shared-log deployment (§2.3) shares the allocation with the workload; in this thesis's implementation that log is ChronoLog, contributing a keeper on every node, a visor coordinating, and a grapher draining sealed chunks to an archive on shared storage. Section 3.3 states which of those details the design actually depends on.

The human in the loop is the *operator*, the researcher or engineer running the fleet, who needs answers both while the system runs and after it finishes:

1. *What is the fleet doing right now?* Which agents exist, which are talking to which, and on which physical nodes?
2. *Is anything failing?* Not as a scroll of per-event errors, but as a ranked, deduplicated set of problems with causes.
3. *Why is this scenario slow?* Which hop in a cross-host delegation chain is the bottleneck?
4. *Which node is dragging the fleet down?* A population-level comparison across all M nodes, including silent ones.
5. *Has this happened before?* Whether a failure is novel or a recurring pattern across runs.

The first three are agent-semantic questions; the fourth is a fleet-topology question; the last is longitudinal, and it is posed because the failure modes of multi-agent LLM systems are empirically systematic rather than incidental [11]: a failure worth diagnosing once is usually worth recognizing again. Answering the middle three together requires pivoting between layers (from a fleet symptom to a single agent's conversation), the pivot neither tooling camp of §2.6 supports.

3.2 Requirements

Seven requirements follow from the questions above; they are the rubric Chapters 6 and 7 check the system against. Three of them (R1, R4, R5) are essential: dropping any one would collapse the thesis's claim rather than degrade it.

	Requirement	What it demands
R1	Dual-plane causal capture	Capture both planes of §2.5.2: LLM turns (prompt, response, model, tokens, cost, latency, retained-context deltas) and inter-agent calls (caller, callee, hosts, timing, outcome). The planes must be *joinable*: an edge resolves to its endpoints' conversations, and every event is attributed to the node that produced it. Without the join, neither the fleet-state question nor the cross-layer pivot is answerable.
R2	Real-time visibility	Live views reflect events within seconds of ingest, not limited by the log store's drain latency, and no read may block the serving process on un-drained log state.
R3	Durability and replayability	Every event survives process and node restarts and stays replayable after the run: at-least-once capture, and retries that do not duplicate in the durable store (effectively exactly-once ingest [27], §2.1.2).
R4	Fleet scale	Population views stay responsive at 100+ nodes: the cost of the fleet overview must not grow with captured events, only with nodes and window length. Silent nodes must appear, since silence is itself a signal: a degraded node that has stopped producing is the fail-slow case that is hardest to see and most damaging [21].
R5	Minimal, non-blocking overhead	Instrumentation must not perturb the workload (the Dapper tenet [49], §2.2.1): the agent-side fast path is a local, constant-time append, all network and log I/O off the critical path, and the machinery degrades gracefully rather than back-pressuring an agent.
R6	Zero-marginal-cost diagnosis	Detection and diagnosis consume no LLM tokens by default; a pipeline that costs money per scan gets disabled in practice. LLM assistance is opt-in on top of a deterministic baseline.
R7	Non-invasive deployment	Attach to an existing deployment without modifying the log store, the scheduler, or the agent framework: a thin client library or local HTTP endpoint, per-node components launchable on a plain SLURM allocation, and a read-only dashboard.

Table 3.1: The seven requirements, derived from the operator questions of §3.1 and used as the evaluation rubric.

3.3 What the Substrate Must Provide

R1–R7 constrain the observability system. They say nothing about *where* the events are stored, and keeping that question separate is what stops the design of Chapter 4 from degenerating into the description of one product. §1.3 commits this thesis to a *class* of store, the distributed shared log of §2.3, but not to a member of that class. This section states the contract between the two: the features S1–S5 that a store must offer for the design to be implementable on it at all, and a second tier S6–S9 of features whose absence costs a known, bounded piece of machinery rather than a redesign.

Required features. Five properties are strictly necessary: a store lacking any one of them cannot host this design without changing what the design *is*.

Convenient features, and the price of their absence. Four further properties are not required, but each one the substrate lacks has to be paid for in machinery. Each is a feature some members of the shared-log class provide and others do not; and for each, this thesis carries a specific, named mechanism that exists only to compensate for its absence in the deployed substrate. Table 3.3 is, read right to left, an inventory of which parts of the design are contributions of the architecture and which are workarounds that a better-provisioned substrate would delete.

	Feature	What it demands, and why
S1	Caller-named streams	A stream is created and opened by an arbitrary caller-chosen string, with no administrative pre-registration and no store-assigned handle the writer must first look up. The whole identity contract of §4.3 rests on this: if the producer can name the stream `writer@scenario-x`, the join between an inter-agent edge and its endpoints' conversations is by construction rather than by a correlation table (R1).
S2	Per-stream append order	Records appended to one stream are read back in the order they were appended. A total order *across* streams is not needed; the design never assumes one, and reconstructs cross-stream causality from explicit correlation identifiers instead (§4.3). This is the weakest ordering guarantee that still supports R3.
S3	Producer-independent durability	Once an append is acknowledged, the record survives the death of the process that wrote it and remains obtainable by a later reader (R3).
S4	Whole-stream replay	A reader can obtain a stream's full history, not merely a subscription to its tail. Every post-hoc view of Chapter 5, and the crash-recovery procedure of §4.4, is a replay.
S5	Reader/writer decoupling	Arbitrarily many independent readers, none of which slows or back-pressures a writer. R5 forbids an expensive diagnostic scan from perturbing the agents being diagnosed, and R4 makes such scans routine.

Table 3.2: The five substrate features required for the design of Chapter 4 to be implementable. Together they describe an append-only, named-stream, replayable, multi-reader store: the shared-log class of §2.3, and nothing narrower.

Candidate substrates. Table 3.4 scores representative members of the class against S1–S9. The systems were built for different purposes and the table does not rank them. It shows that the required tier S1–S5 is satisfied broadly, so the design is portable, while the convenient tier S6–S9 is where the systems differ, so the design's portability costs are predictable from the table rather than discovered during a port.

The last observation is the reason ChronoLog is the substrate of this work rather than an incidental choice of one. S6 is the feature that couples agent semantics to physical topology for free, which is the gap of §2.6.3; no general-purpose commit log in the table offers it, because none was built to run inside an HPC allocation. The design takes S6 and pays for the absence of S7–S9, and Chapter 4 is written so that the payment is separable from the design it is paying for.

3.4 Non-Goals

Four adjacent problems are explicitly out of scope. The system is not an *agent framework* (it observes agents built with existing SDKs and takes no part in orchestrating them), not a *scheduler or resource manager* (placement and job control remain SLURM's; the system only queries it), and not a general *metrics time-series database* (it captures the agent-semantic planes of R1, not arbitrary node telemetry, so where classical infrastructure metrics are needed the systems of §2.6.2 remain the right tool and the two run side by side). Finally, it targets observation and diagnosis, not automated remediation: acting on a diagnosis is surfaced as a recommendation, not executed autonomously.

With the questions, requirements, substrate contract, and non-goals fixed, the problem is fully specified. Chapter 4 presents a design that meets R1–R7 on any store satisfying S1–S5, and reports separately, in §4.8, what implementing it on ChronoLog required; Chapter 6 then measures each requirement against that implementation.

	Feature	If absent, the cost	Mechanism that pays it
S6	Node-local append endpoint: a write path reachable without leaving the producing node	Appends cross the network, so the producer's fast path must be buffered locally to satisfy R5, and the producing host must be stamped by the client rather than inferred from where the append landed	The node-local spool, §4.4 (retained regardless, so the marginal cost is only the explicit host stamp)
S7	Stream enumeration: ask the store which streams exist	No listing screen can be built, since a stream can be read only if its name is already known	The `__index__` stream: materialize the missing query as another log, §4.3
S8	Bounded, non-blocking read visibility: a freshly appended record becomes readable within a small bound, and a read that finds nothing returns rather than waits	Live views cannot be served from the store, and a read issued into the gap can stall the process that issued it	The hot/cold split and live bus, §4.5; the process isolation rules of §4.6
S9	Positional reads: resume a stream from a recorded offset	Every read is a full replay, so incremental readers must track their own progress and re-scan	Reader-side high-water marks, §4.4; pre-aggregated rollups so that fleet reads never scan raw history, §5.3

Table 3.3: Substrate features whose absence is survivable, with the mechanism this thesis carries to compensate for each. The right-hand column is the portion of Chapter 4 that a substrate providing S6–S9 would let a reimplementation delete.

	Required					Convenient			
Substrate	S1	S2	S3	S4	S5	S6	S7	S8	S9
Kafka [30]	✓	✓	✓	○	✓	×	✓	✓	✓
Pulsar [4]	✓	✓	✓	✓	✓	×	✓	✓	✓
Redis Streams [46]	✓	✓	○	✓	✓	○	✓	✓	✓
CORFU/Delos [7, 6]	○	✓	✓	✓	✓	×	×	✓	✓
BookKeeper [25]	○	✓	✓	✓	✓	×	✓	✓	✓
ChronoLog [28]	✓	✓	✓	✓	✓	✓	×	×	×
Local JSONL files	✓	✓	○	✓	✓	✓	✓	✓	✓

Table 3.4: Representative shared-log substrates against the feature tiers. ✓ = provided, ○ = provided with caveats, × = absent. Caveats: Kafka's whole-stream replay is bounded by the configured retention policy; Redis Streams and local files give durability weaker than S3's cluster-wide sense (process-local rather than node-independent); CORFU and BookKeeper expose a shared address space and ledgers respectively, on which named streams are a layer built above rather than a native primitive. The ChronoLog row is scored for the deployed version and client binding used in this work (§4.8), not for the design in the published paper. The last row is not a shared log at all but the degenerate single-node case, included because it is the offline adapter of §4.7 and because it shows which features are hard only *because* the store is distributed. Two observations drive the rest of the thesis: every row satisfies the required tier, subject to the caveats above, and ChronoLog is the only general-purpose log store here that provides S6, the property that makes agent events natively host-attributed, while also being the only one missing S7–S9, which is exactly the machinery Chapter 4 must supply.

Chapter 4

System Design

This chapter presents the design of the framework, written against the substrate contract of §3.3 rather than against any one log store: §§4.1–4.7 assume only the required features S1–S5, and name at each point which of the convenient features S6–S9 the mechanism under discussion is compensating for. Section 4.1 gives the architecture, §4.2 the three substrate gaps that shape the rest, §4.3 the mapping of agent semantics onto the log's data model, §4.4 the write path and its delivery guarantees, §4.5 read visibility and the hot/cold split, §4.6 process and failure isolation, and §4.7 the extensibility seam. Section 4.8 then collects everything true of ChronoLog in particular: how the generic model maps onto its data model, what its deployed client binding did and did not provide, and what a port to another substrate would add or remove.

4.1 Architecture Overview

Chapter 3 fixed what the system must do; this chapter describes how it is built. The setting is the one of §3.1: a SLURM allocation of compute nodes, several LLM agents running as independent processes on each, delegating sub-tasks to one another across nodes, and an operator who needs to see what the fleet is doing without slowing it down. Three kinds of component are added to that picture: a capture library and a per-node daemon that record what the agents do, a storage layer that holds the record, and a dashboard (a Flask application serving a React single-page app) that reads the record back. This section describes how data reaches storage; §4.5 describes how it is read back out.

The storage layer is a distributed shared log (§2.3): an append-only store of named streams in which producers append at the tail and any number of readers consume independently, at their own pace, without slowing the producers down. What matters here is that a log of this class is deployed *across* the cluster rather than beside it. A process of the log store runs on every compute node and accepts appends from the processes on that node, buffering them in memory; asynchronously, a separate pipeline drains those buffers into a durable, time-ordered archive on shared storage. In the implementation used here that store is ChronoLog (§2.4), its per-node component is the *keeper*, and its deployment shares the operator's SLURM allocation with the agents being observed. Two properties of that arrangement shape everything below: an append never leaves the machine that issued it, so capture costs an agent a local write rather than a network round trip; and the node an event came from is known from where it was appended.

Every datum enters the system through one of two ingest paths (Figure 4.1), both terminating in the shared log through the append endpoint on the producing node.

Path-A carries the intra-agent plane: LLM interactions (one record per turn), retained-context deltas (one record per addition or eviction from the agent's retained context), and recovery events. Producers call a small capture library that appends the record to a node-local, append-only *spool*; a separate *capture worker* process periodically drains the spool into the log.

Path-B carries the inter-agent plane: agent-to-agent MCP calls. Each logical call is emitted as a *pair* of events sharing a correlation identifier: a `start` when the delegation is issued and a `done` when it completes, the latter carrying status and latency. Agents POST these events to a *collector* daemon on their own node (`localhost:5650`); the collector appends them to the log through the node-local endpoint and simultaneously forwards a thin copy to the dashboard, which feeds the live views (§4.5).

The two paths differ because their traffic differs. Path-A events are large (full prompts and responses), produced at low rate (one per turn, i.e. per LLM call), and consumed mostly after

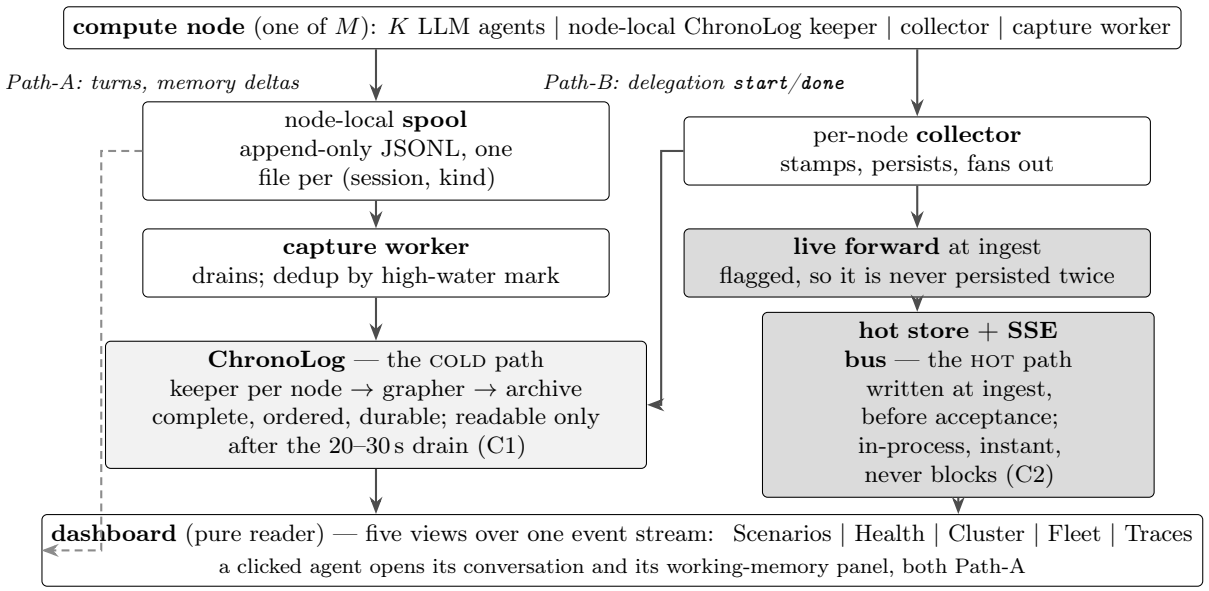


Figure 4.1: The two-path architecture. Path-A carries LLM turns and retained-context deltas through a node-local spool drained by a capture worker; Path-B carries inter-agent calls through a per-node collector. Both persist to the shared log through its node-local append endpoint, drawn here as the ChronoLog keeper of the deployed instantiation (§4.8). The dashboard reads the log (cold, complete, late) and the hot store (live, written at ingest), and renders five views over the one stream; the dashed edge is the spool doubling as Path-A's hot store, read directly by the Health and context panels.

the fact, so a durable local buffer plus a batch drain fits. Path-B events are small, structurally fixed, and latency-relevant (the live scenario graph is drawn from them), and they define the cross-agent causal structure; a daemon that stamps, persists, and immediately fans them out is the better match. What the paths share is the destination and the discipline: node-local append on the producing machine, which keeps capture off the agent's critical path (R5, and S6 where the substrate offers it), durable persistence in the log (R3, via S3), and host attribution stamped at source (R1).

The dashboard is strictly *read-only*: it reconstructs every view from the log and the hot store, and captures nothing on its own (§4.6).

4.2 Three Gaps in the Deployed Substrate

Adopting a shared log as the substrate delivers what Chapter 2 described: node-local appends, decoupled readers, durable order, and host attribution by construction. Its costs fall on the read side. The deployed substrate satisfies the required tier S1–S5 in full; what it does not provide is S7 (enumeration), S8 (bounded, non-blocking read visibility), and S9 (positional reads). Three concrete symptoms, two of the missing S8 and one of the missing S7, shaped nearly every subsequent decision in this chapter, and because the same three recur across the design, the evaluation, and the future work, they are stated once here and referred to as C1–C3 thereafter. The missing S9 produces no comparable symptom: it costs the reader a full replay instead of a resumable one, paid for by the high-water marks of §4.4 and the pre-aggregated rollups of §5.3 rather than by anything structural.

The three gaps are not of the same kind. C1 is a consequence of the write-optimized design point of §2.3 and would appear in some form on *any* log of this class; it is a property of the architecture, and the response to it belongs to the design. C2 and C3 are properties of the deployed client binding rather than of the published substrate design, and §4.8 reports them as such; the responses to them are workarounds, and §7.2 records that both are being removed upstream.

C1: reads trail writes by a visibility window. Appends land in the node-local ingest buffer and return immediately; the log store drains sealed chunks to read-optimized storage asynchronously. Between those two moments an event is durable but not yet readable: measured end-to-end at 20–30 s in the deployments used here (§6.3). ChronoLog names this interval the *acceptance window*, and the rest of the thesis uses that term for the concrete quantity and *visibility window* for the generic property. Every live view the operator asks for falls inside exactly that window, because live views show what *just* happened. This is the standard write-optimized trade-off, not a defect, and it is answered by the hot/cold split of §4.5: the log is the durable cold path, and a hot path populated at ingest serves the window.

C2: a read into the window blocks, and blocks everything. C1 would be a tolerable staleness problem on its own. What makes it an architectural constraint is the failure behavior of the read that hits the window. A read the log store cannot answer does not return stale data or an error: it waits. In the deployed binding the wait is not merely long (∼180 s to the client's query timeout) but takes the entire process with it, because the binding holds Python's global interpreter lock for the duration; §4.8 gives the mechanism. The boundary case is worse than the common one: a read of a stream that has never been written blocks not for the timeout but indefinitely, since there is no data in flight that could ever arrive, which is precisely the state of a freshly deployed cluster. Two design rules follow, and neither is optional: no process that must always answer may share a failure domain with one that may legitimately block (§4.6), and reads that risk touching un-drained state are served archive-first, where absence is detectable instantly (§4.5). The evaluation later found that live replay does not reach the dashboard's receiver at all in these deployments, making *every* dashboard replay hit this path (§6.8); the rules written for the boundary case turned out to cover the general one.

C3: the log has no catalog. The client binding exposes no operation to enumerate what the store holds, and therefore no way to ask "which scenarios exist?" or "which sessions were captured?" without already knowing their names. A stream can be read only if its name is known. This is not a performance property but a missing query, and it blocks the dashboard's most basic screen, the listing every view starts from. The response is to materialize the missing query as another log (§4.3): an `__index__` namespace to which every component appends the name of each scenario or session it sees for the first time, so that listing becomes a replay of one well-known stream rather than an enumeration the log store cannot perform.

The three compound: C3 forces a read of the index stream, C1 means that stream may be un-drained, and C2 means the resulting block takes the dashboard down with it. Read together they explain why the read path of §4.5 looks more defensive than a log-backed dashboard would otherwise need to be. They are also the part of this design least likely to outlive it: §7.2 reports that the C2 and C3 gaps are being closed upstream, and §4.7 isolates all three behind an interface so that a substrate without them costs an adapter, not a redesign.

4.3 Mapping Agent Semantics onto the Log

The substrate contract of §3.3 supplies one structural primitive: caller-named streams of ordered records (S1, S2). Every shared log of the class layers a two-level namespace over it, under different names: ChronoLog calls the levels *chronicle* and *story* (§2.4.2), Kafka calls them *topic* and *partition* [30], Pulsar *namespace* and *topic* [4]. This section uses the neutral terms *namespace* and *stream*; the ChronoLog spelling is given in §4.8.

The mapping adopted here (Table 4.1) uses namespaces to partition by *data kind* and streams by *entity*, so that every read pattern of Chapter 5 replays exactly one stream:

Namespace	Stream	Holds	Path
`llm_interactions`	*session id*	LLM turns	A
`context_graphs`	*session id*	retained-context deltas	A
`recovery_events`	*session id*	recovery/backtrack events	A
`ia_<scenario>`	`edges`	inter-agent `start`/`done` events	B
`metrics_rollup`	`minute`	per-(host, minute) fleet buckets	rollup
`__index__`	`scenarios` / `sessions`	listing index	both

Table 4.1: The namespace/stream schema: namespaces partition by data kind, streams by entity. Session-scoped streams serve the conversation and context reads; the per-scenario edge stream serves graph and trace reads; the rollup and index streams are written by the capture side so that readers never scan. In the ChronoLog instantiation the two levels are chronicles and stories respectively (§4.8).

Session identity. The central convention is the session identifier `role@scenario` (e.g. `writer@scenario-x`). It is simultaneously the stream name for all Path-A data of that agent, the `from_session`/`to_session` endpoint on Path-B edges, and the join key the dashboard uses when a clicked node in the scenario graph opens that agent's conversation. Scoping the role by scenario keeps the identifier globally unique across concurrent scenarios while leaving it human-readable. Because the same string names the spool file, the log stream, and the graph node, the identity round-trips through every tier with no lookup table. This is R1's "joinable planes" made concrete: the join is by construction, not by correlation heuristics, and it is the one place where the design leans on S1 rather than merely benefiting from it. A substrate that assigned stream handles instead of accepting caller-chosen names would force a name-to-handle table on every writer and reader, and the join would become a lookup that can be stale.

Causal structure. Cross-agent causality is carried by two explicit fields on Path-B events, not by the log store: S2 guarantees order only *within* a stream, so nothing in the design assumes a total order across streams. The `correlation_id` stitches the `start`/`done` pair of one call into a single edge carrying both endpoints, both hosts, the outcome and the latency. That reassembled object is called a *stitched edge* throughout the rest of the thesis, and it is the unit every inter-agent view is built from: the scenario and cluster graphs draw them, and the tracer treats each one as a span (Chapter 5). The optional `parent_correlation_id` records the call being *serviced* when a new call was issued, giving the tracer exact call trees. When instrumentation does not propagate the parent, the tracer falls back to inference from session identity and time nesting (§5.4), the Mystery-Machine position on the spectrum of §2.2.1, with explicit propagation as the Dapper position when available. Within a session, Path-A records carry a monotonic per-stream `sequence_id`, which orders turns without reference to wall clocks and, as §4.4 shows, doubles as the reader-side substitute for the offsets a substrate with S9 would supply.

Listing without enumeration. C3 (§4.2) leaves the system unable to ask which scenarios or sessions exist. The `__index__` namespace of the table is the answer: every component that first sees a new scenario or session appends a one-line event (`{"v": <name>}`) to a well-known index stream, and listing becomes a replay of that stream, deduplicated in first-seen order. The index is append-only and idempotent to re-add, so it needs no coordination among the writers, and it costs one event per entity rather than one per observation. The pattern is a small instance of the log-as-substrate principle [29]: when a query is missing, materialize it as another log. On a substrate providing S7 the index is redundant for listing and survives, if at all, only as a cheap first-seen ordering.

4.4 Write Path and Delivery Semantics

The write path is engineered backwards from R5 (never perturb the agent) and R3 (never lose, never duplicate).

The spool as write-ahead buffer. A Path-A producer's fast path is an append of one JSON line to a per-(session, kind) spool file, opened with `O_APPEND` so that concurrent writers interleave atomically at line granularity [24]. No network, no lock, no dependency on any downstream component being alive: if the log, the capture worker, or the dashboard is down, agents keep running and the spool absorbs the backlog. The arrangement is the write-ahead log of database recovery [37] reduced to its minimum: an append-only file that is cheap to write and replayable after a crash, in the tradition that treats sequential appends as the cheapest durable write a system can make [47]. The capture library injects and guarantees exactly three fields: `session_id`, a monotonic `sequence_id` (seeded from the maximum found on disk, so monotonicity survives producer restarts), and a `timestamp`. It treats the rest of the record as an opaque producer contract. The spool is what makes S6 a preference rather than a requirement: it already decouples the producer from the append, so a substrate with no node-local endpoint costs the design only an explicit host stamp, not a new buffering tier.

Effectively exactly-once ingest. The capture worker drains the spool into the log on a polling interval. Transport is at-least-once by construction (after a crash the worker re-reads spool files it may have partially drained), so ingest must deduplicate. The worker keeps a *high-water mark* per (session, kind) stream: the largest `sequence_id` already written to the log. Only records above the mark are appended. The critical property is where that state lives: on startup the worker *rebuilds the marks from the log itself*, replaying each session's streams to find the highest sequence present. The log is thus the single source of truth for what has been ingested, and the worker carries no durable state of its own. No worker crash, restart, or migration to another node can cause loss (the spool retains everything) or duplication (the rebuilt marks skip everything already present). Records without a natural sequence (recovery events) carry a unique blob identifier and are deduplicated by set membership, seeded the same way. The arrangement is the textbook recipe of §2.1.2 (at-least-once transport plus idempotent, monotonic ingest [27]), instantiated with the log as its own ledger. It is also where the absence of S9 is paid for: a substrate with positional reads would let the worker persist and resume from an offset, as a Kafka consumer does with its committed position [38], instead of replaying a stream to rediscover where it left off. The rebuild-from-the-log variant is strictly more robust (there is no offset store to lose) and strictly more expensive (it is $O(\text{stream})$ per restart, not $O(1)$), and it was chosen because the log store left no alternative.

Path-B: persist once per backend. The collector gives Path-B events the same guarantees with one extra subtlety (Figure 4.2). Every event travels to *two* places: the log (durable) and the dashboard (for the live bus). Naively this double-writes. The design makes the forward explicit: when, and only when, the collector has successfully written an event to the log, it sets a header (`X-DTP-Forwarded: chronolog`) on the forwarded copy. The dashboard honors the header by preserving the collector-stamped identifiers verbatim and skipping its own durable write, storing only the hot copy. If the durable write failed, the collector forwards *without* the header and the dashboard persists the event through its own backend; the placement is degraded, but the event is not lost. The net contract: each event is persisted exactly once per backend, and the identifiers (`event_id`, timestamps) are identical in both copies, so the live view and the durable view never disagree about identity.

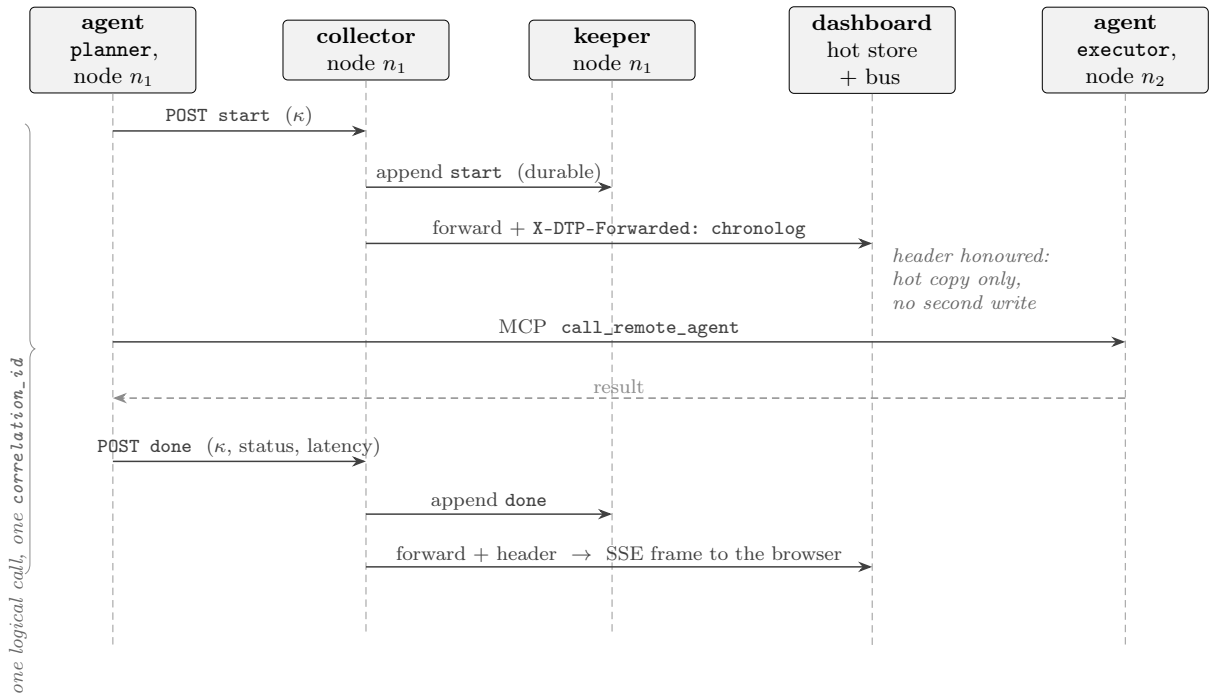


Figure 4.2: One inter-agent call, end to end. The `start`/`done` pair shares a `correlation_id` (κ), so the two events stitch into a single edge. Each event travels to two places: the node-local keeper (durable) and the dashboard (live). The collector sets `X-DTP-Forwarded` on the forwarded copy *only* when the durable write succeeded, and the dashboard honours it by keeping a hot copy and skipping its own write, which is what makes the delivery exactly-once *per backend* rather than duplicated across them. If the keeper write fails the header is omitted, the dashboard persists the event through its own backend, and the placement degrades without the event being lost (§4.4).

Graceful degradation. One more fallback closes the path: an agent's `emit()` tries the node-local collector first and falls back to POSTing directly to the dashboard's ingest endpoint if the collector is down. The fallback sends no forwarded header, so the dashboard persists normally. An agent therefore observes at most a failed local HTTP call before falling back: never a lost edge, and never back-pressure from the log (R5). Each degradation step trades a property the operator can lose (placement, freshness) for one they cannot (the event itself), which is the standard stability-pattern discipline of keeping a failure in one component from propagating into its callers [39].

4.5 The Read Problem: Hot and Cold Paths

Writing to an ingest-optimized log is easy; reading it live is the hard part. C1 and C2 together (§4.2) rule out the obvious design: a dashboard that replayed streams on request would serve nothing for the first half-minute of a stream's life and would freeze outright, for minutes at a time, precisely while the fleet is live. That is a direct violation of R2. Everything in this section is the price of the missing S8; on a substrate that provides it, the cold path alone would serve every view.

The design response is a lambda-style split (§2.3) with the visibility window as the boundary [35]:

The cold path is the log itself: complete, durable, ordered, and authoritative, but late. Full-fidelity views (a session's complete conversation, post-hoc analysis) read it, either by live replay once a stream has drained or from the archive. Reads that risk touching un-drained state are avoided structurally: the listing index of §4.3 is read *archive-first*, because a live replay against a never-written index stream blocks indefinitely (C2). The evaluation later extended this rule into

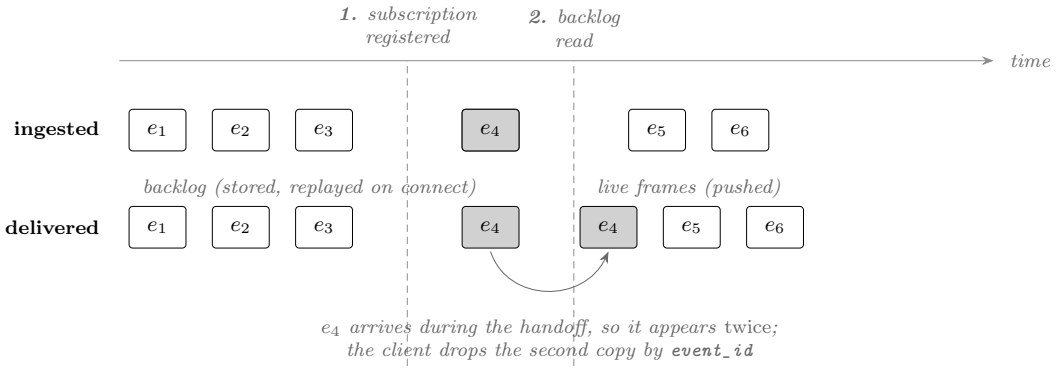


Figure 4.3: The late-join protocol. A client connecting mid-run is subscribed to the live bus *before* its backlog is read, so any event ingested during the handoff is delivered on both routes. The ordering is deliberate: it converts a possible gap, which the client cannot detect or repair, into a possible duplicate, which it removes by `event_id`.

a deployment mode: after live replay proved unreliable end-to-end on the cluster, archive-first reading was generalized from the listing index to *every* replay the serving processes issue, bounding staleness at the drain window (§6.3, §6.8).

The hot path serves R2 and is populated *at the moment of ingest*, before the durable write is visible. It has two elements. First, a *hot store*: the ingest endpoint appends every Path-B event to a local JSONL file per scenario, and all live read APIs (the scenario graph, the node-communication view, fleet aggregation, the tracer) read this store, an instant, append-only local read that can never block on the log. Second, a *live bus*: an in-process publish/subscribe [17] fan-out that pushes each newly ingested event to server-sent-event subscribers [51], so the browser renders edges as they happen rather than polling. Path-A has the analogous arrangement: the Health view and the per-agent context panels read the capture spool directly (the spool doubling as Path-A's hot store) while the log remains the durable copy.

The late-join protocol. A client that opens a live stream mid-run needs history plus continuity without gaps or duplicates (Figure 4.3). The bus protocol delivers the stored backlog first, then live frames, and the subscription is registered *before* the backlog is read, so an event ingested during the handoff appears in both, and the client deduplicates by event identifier. Delivery to subscribers is intentionally weaker than delivery to storage: per-subscriber queues are bounded and drop oldest under pressure, and a publish never blocks or raises into the ingest path. By the time an event reaches the bus it is already durable, so the live stream is a best-effort projection of a reliable log, not a second reliability domain (R5).

Consistency model. Each view advertises, in effect, one of two freshness classes: hot views are eventually consistent with the log but fresh to within the ingest latency (measured in Chapter 6); cold views are complete and ordered but stale up to the visibility window. Because both are projections of the same event stream with identical identifiers (§4.4), the classes converge: after the drain, a hot view's content is a prefix-complete subset of the cold view's, and the operator can always escalate from "live picture now" to "authoritative record later" without changing tools.

4.6 Process and Failure Isolation

The deployment decomposes into single-responsibility processes per node (agents, collector, capture worker), plus one dashboard for the allocation. Two isolation rules, each learned from a concrete failure on live clusters, govern the decomposition.

The dashboard is read-only. This is C2 turned into a deployment rule. An early configuration ran the capture worker as a thread inside the dashboard process; on startup the worker warms its high-water marks by replaying sessions (§4.4), and on a cluster where a stream sat inside the visibility window that replay blocked, taking the whole interpreter with it (§4.8), so the Flask server never finished binding its port. The two now run as separate processes, and the rule extends to reads: no request handler ever issues a replay that could touch un-drained state. Correspondingly, a brand-new deployment seeds the index with a one-shot append-only sync pass rather than a blocking warm-up, so the never-written boundary case of C2 is never reached. Stated substrate-independently: a component with a liveness obligation must not share a failure domain with a call whose worst case is unbounded. The rule is worth keeping even on a substrate that honours S8, because it costs one process and removes an entire category of outage.

Normalize identity at the edge. Host names arrive from multiple authorities (reverse DNS reports `ares-comp-3`, SLURM reports `ares-comp-03`), and any view that joins on host names (fleet health, the cluster graph, allocation membership) silently drops nodes if the two spellings coexist. All host names are therefore canonicalized (digit padding stripped) at every boundary where they enter a data structure. Identity normalization, like time, belongs at the edge of the system, not in each consumer.

Failure containment follows from the decomposition: an agent crash loses nothing already spooled or emitted; a collector crash falls agents back to direct dashboard ingest; a capture-worker crash pauses draining while the spool absorbs; a dashboard crash affects only observation (capture continues); and a log outage degrades the system to hot-store-only operation, with the spool and collector queues replaying into the log when it returns.

4.7 Extensibility: Adapters and Capabilities

Every dashboard view is declared as a *capability* (conversations, scenarios, diagnostics, fleet, tracing, . . .) served by a *source adapter*, an interface discovered at runtime through a Python entry-point group. The shipped ChronoLog adapter serves all capabilities; at startup the application discovers active adapters, mounts only the API blueprints some adapter can serve, and advertises the active set to the frontend, which renders only those tabs. The seam exists for third parties: a deployment with a different substrate implements the adapter interface and gains the dashboard unchanged. The same seam is what makes the offline demo mode (local stores, no cluster) a configuration rather than a fork, and the "local JSONL files" row of Table 3.4 is exactly that mode's substrate.

The contract an adapter must honour is S1–S5, and nothing more. What S6–S9 an adapter can additionally offer determines how much of this chapter its deployment executes rather than what it must reimplement:

- An adapter over a substrate with S8 declares its reads bounded and non-blocking; the hot store and live bus of §4.5 become a latency optimization rather than a correctness requirement, and the archive-first rule of §4.6 is dropped.
- An adapter over a substrate with S7 answers listings natively; the `__index__` writes of §4.3 are disabled.
- An adapter over a substrate with S9 persists reader offsets; the high-water-mark rebuild of §4.4 becomes an $O(1)$ resume.
- An adapter over a substrate without S6 stamps the producing host in the capture library rather than inheriting it from the append endpoint.

This is why the chapter is separable. The capture paths, the identity contract, the delivery semantics, and the two-plane schema of §§4.3–4.4 are the design, and they hold on any S1–S5

substrate. The hot/cold split, the index stream, and the reader-side high-water marks are the price of S7–S9 being absent here, and a different deployment pays a different price. The analytics of Chapter 5 depend only on the capability interface and therefore on neither.

4.8 Instantiation on ChronoLog

This section reports what the design above became when implemented on ChronoLog (§2.4), which is the substrate every measured result in Chapter 6 was obtained on. Nothing here is required by the design; everything here is required to reproduce the deployment.

Names. ChronoLog's two-level namespace is *chronicle* → *story*, with *event* as the record (§2.4.2). The generic "namespace" column of Table 4.1 is a chronicle and the "stream" column is a story; `llm_interactions` is a chronicle holding one story per session, `ia_<scenario>` a chronicle holding a single `edges` story. Stories are the unit of isolation, so writers to different sessions do not contend, which is what makes the per-entity stream granularity of §4.3 free rather than a partitioning decision.

Where the append endpoint is. ChronoLog supplies S6 directly: a *keeper* process runs on every node of the deployment, and a client appends to the keeper on its own machine (§2.4.1). This is the property that motivated the choice of substrate. Because the append lands on the producing node, the mapping from an event to the machine that generated it is a fact about where it was written rather than a field the instrumentation must be trusted to fill in correctly, and R1's host attribution is structural. The capture worker and the collector are therefore deployed one per node, each holding a client bound to the local keeper.

C1 concretely. The keeper buffers appends in memory and the *grapher* asynchronously drains sealed chunks into a read-optimized archive (CSV/HDF5 tiers) that the *player* serves back to readers. In the deployments used here the keeper seals 10 s chunks under a 15 s acceptance window, while the grapher and player run with a 180 s window; §6.3 measures the end-to-end consequence at 20–30 s and corrects a folklore figure in the process.

C2 concretely. A replay the player cannot answer waits for the client's ∼180 s query timeout, and the Python binding holds the global interpreter lock (GIL) for the duration. The GIL admits only one thread executing Python at a time; a well-behaved extension releases it before a slow wait, and this one does not. The consequence is that a single unlucky read freezes not a request but the *entire interpreter*: every other thread in that process, including the web server's accept loop. This is a property of the binding, not of ChronoLog's design, and it is the reason §4.6's read-only rule is stated as a process split rather than a thread pool.

C3 concretely. The client binding exposes no `ShowChronicles` operation, so there is no call that enumerates the chronicles or stories a deployment holds. This is a missing method rather than a performance property, which is why §4.3 answers it with data (the `__index__` chronicle) rather than with caching.

Deployment. A run consists of a ChronoLog deployment across the SLURM allocation (visor, one keeper per node, grapher and player draining to an archive on shared storage), plus one collector and one capture worker per node and one dashboard for the allocation (Figure 4.4). Nothing in ChronoLog, SLURM, or the agent SDK is modified, which is R7; §6.6 verifies this end-to-end.

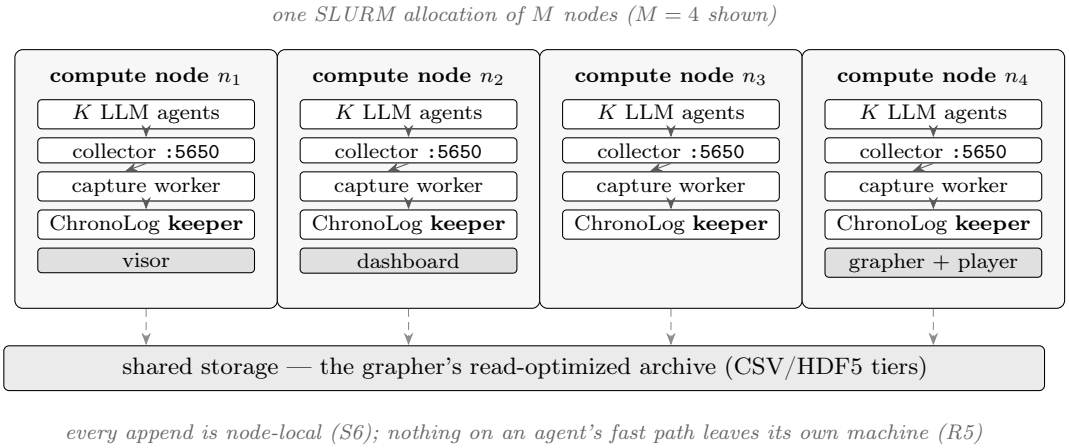


Figure 4.4: What runs where. Every node of the allocation carries the identical per-node set (agents, collector, capture worker, ChronoLog keeper); only the singletons differ, and their placement follows the deploy convention (visor on the first node, grapher and player on the last, dashboard on the second). Drawn from the four-node deployment used for the recorded walkthrough. The layout is the point: because a keeper sits on every node, the agent's append never leaves the machine that produced it, which is S6 and the reason host attribution is structural rather than declared.

What a port would change. Table 3.4 predicts the cost of moving to another substrate, and the prediction is concrete enough to state. Porting to Kafka would delete the hot store, the live bus's role as a correctness mechanism, the `__index__` chronicle, and the high-water-mark rebuild, since Kafka provides S7–S9; it would add an explicit host stamp in the capture library and a per-node producer buffer to preserve R5, since it does not provide S6, and it would add a retention policy to preserve S4's whole-stream replay. Porting to the local-JSONL adapter, which the offline demo mode already exercises, deletes all of the same machinery and adds nothing, at the cost of S3's cluster-wide durability. What survives every port is the two-path capture architecture, the `role@scenario` identity contract, the two-plane schema, and the delivery semantics, which is the claim §4.7 makes and the reason the design is stated against S1–S9 rather than against a product.

Chapter 5

Fleet-Scale Analytics on the Log

Chapter 4 produced a complete event record: every observable event of the fleet, durably ordered and attributed to node and agent, readable either from the log itself once it has drained (the cold path, complete but tens of seconds late) or from the ingest-time hot store that serves the interval before that (§4.5). This chapter presents the analyses that turn that record into answers to the operator questions of §3.1. Each section follows the same shape: the question, the algorithm, and the argument for why it stays cheap at fleet scale. The screenshots throughout are captures of the deployed system; a recorded walkthrough of the same operator surface, on a live four-node cluster running seven agents, is available online.[1]

5.1 Live Topology Coupling: Scenario Graph and Cluster Graph

The first operator question is situational awareness: what is the fleet doing, right now, and where. The system answers it with two live graphs, built as *two projections of the same event stream.*

Both are built from the inter-agent plane, Path-B of §4.1: every delegation between agents was captured as a `start` and a `done` sharing a correlation identifier, and reassembled into one *stitched edge* carrying both endpoint sessions, both hosts, the outcome, and the latency (§4.3). Being live, both graphs read the *hot store* rather than the log: the ingest-time copy that §4.5 introduced precisely because the log itself cannot be read inside its visibility window.

The *scenario graph* projects those edges onto agent identity: nodes are sessions (`role@scenario`), edges are stitched calls labeled with tool, status, and latency, built incrementally in the browser from the *live bus*, the publish/subscribe fan-out of §4.5 that pushes each newly ingested event to the browser: the stored backlog first, then live per-event frames. LLM-observability tools already offer this view (§2.6.1); here it is rendered live. The *cluster graph* projects the very same edges onto physical topology: hosts as nodes, aggregated host-pair traffic as edges (counts, error counts, active agents per host), the projection no Camp-1 tool can perform because host identity is not a dimension it groups over (§2.6.1), computed by folding the hot store. Because every edge carries `from_host` and `to_host` stamped at capture (R1), the projection is a group-by, not an inference.

The coupling is completed by the scheduler's view of the machine: the dashboard queries SLURM (`sinfo`) for the allocation and the cluster's total/idle/busy/down states, draws idle-but-available nodes alongside the active deployment, and uses only real node names throughout. A node with no events is drawn grey, not omitted; at fleet scale, silence is a signal (R4). Clicking through connects the layers: an agent node opens that agent's conversation (the `role@scenario` join of §4.3); a host cell pivots to the per-node view. This two-projection design (same events, agent layer and machine layer) is the direct answer to the gap of §2.6.3.

5.2 Failure Detection, Clustering, and Diagnosis

The second is not "show me the errors" but "tell me what is wrong": at fleet scale, a per-event error feed is least readable when it matters most. This section describes the analysis built for that question: a five-stage pipeline (*gather* → *detect* → *cluster* → *diagnose* → *record*) that turns raw events into a ranked list of explained incidents. It is a component of the system presented

[1] Video walkthrough: `https://youtu.be/H_CoekzZsks`. Recorded on a four-node allocation of the ARES cluster with seven agents placed three/two/one/one across the nodes, delegating over MCP.

here, not an external tool; the dashboard exposes it as the *Doctor* tab, and the remainder of the thesis calls it the Doctor. The shape is the one established by console-log mining for large systems [53]: normalize messages into templates, group by template, and present groups rather than lines; Oliner et al. [40] survey the wider design space that shape sits in. What differs here is the input, which is not free-text logs but a structured, causally ordered event stream, so the detectors can key on typed fields rather than on parsed text.

Gather and detect. Gathering is the only I/O, and it reads only hot sources: interactions and recoveries from the Path-A spool, stitched edges from the Path-B hot store, with explicit bounds on sessions and scenarios scanned. Neither read touches the log, which matters because a scan runs inside the dashboard process: a replay that landed in the visibility window would block that process outright (C2, §4.2), taking the whole interpreter with it. Detection is a set of pure, unit-tested functions over the gathered lists, each emitting typed *signals*: HTTP 4xx/5xx and explicit errors on turns (429 classified separately as rate-limiting); latency outliers, flagged relative to the *per-model* median (an absolute threshold would misclassify a slow model as an incident); retry storms (the same prompt re-issued repeatedly within a session); failed inter-agent calls; *orphan calls* (a `start` with no `done`, the signature of a hung or dropped delegation); slow edges relative to the per-tool median; and recovery events. Orphan calls are always ranked critical: the delegating agent is silently stalled, and no per-edge view shows anything wrong. Detection is a single pass: $O(n)$ in gathered events, with per-group medians over small per-model and per-tool groups.

Clustering. The view scales because fleet-wide failures are *textually self-similar*. Each signal's message is normalized, with identifiers, hex runs, and digit strings replaced by placeholders (`peer ares-comp-12 timed out after 30001ms` → `peer ares-comp-<n> timed out after <n>ms`), and the pair (kind, normalized message) is hashed into a *signature*, the clustering key. Normalization by placeholder substitution is the fixed, hand-specified end of the log-parsing spectrum whose learned end is occupied by online template miners such as Drain [22]; the fixed variant is chosen here because R6 forbids a per-scan cost and because empirical comparisons of log-analysis pipelines find that parsing quality, not detector sophistication, dominates end-to-end accuracy [23], and the message set here is generated by our own detectors rather than by arbitrary third-party code. Five hundred identical timeouts across the fleet collapse into one incident with a count of 500; hosts, sessions, tools, models, and scenarios are aggregated *inside* the incident as its blast radius, deliberately excluded from the key so that a fleet-wide failure stays one card while its distribution stays visible. Severity escalates with count, and incidents rank by (severity, count, hosts affected), worst first.

Diagnosis. Each incident gets a diagnosis (root cause, remediation, blast radius, confidence) from a deterministic, first-match-wins rule engine keyed on signal kind and sample text: orphan calls map to hung-peer causes, 429s to rate limiting, transport-layer markers to network-fabric issues, token-overflow patterns to context exhaustion, and so on, each with a calibrated confidence; a conservative fallback catches the rest. Attributing a failure to a responsible component is the problem Pinpoint [12] posed for internet services, and it solved it by correlating the success or failure of many requests with the components each one touched. The setting here is easier in one specific respect: causal attribution is already explicit in the events (§4.3), so what remains is naming the cause, not locating it.

The engine costs nothing per scan (R6). The learned alternative, sequence models over log streams of which DeepLog [16] is the canonical instance, generalizes to anomalies no rule anticipates, but buys that with a training corpus and a per-event inference budget: exactly the per-scan cost R6 forbids. An LLM-backed diagnoser can be enabled explicitly, and any failure of it falls back to the heuristic: the deterministic path is the contract, the LLM an enhancement.

Recording recurrence. The final stage answers whether the failure has happened before. Every scan appends one line per incident to a Markdown incident history file; when the recent section exceeds a threshold, the oldest lines fold into a digest with running counts; the file is *self-compacting*, so it stays small across arbitrarily many runs while retaining per-signature totals. On each scan, current incidents are matched against remembered signatures, and a repeat offender is badged `recurring` $\times N$. The history is a plain human-readable file: the operator can read, annotate, or delete it, and it survives restarts by construction.

5.3 Fleet Health at Scale

The fourth question is population-level: which node is unlike the others. The node-link rendering of §5.1 answers it only up to a dozen nodes; past that, it is spaghetti. The fleet view therefore changes representation (a heatmap grid, one cell per node, colored by health, worst first) and, more fundamentally, changes the *read complexity*. The representation change follows the overview-first, then zoom-and-filter, then details-on-demand discipline [48]: the grid is the overview that must stay legible at population scale, and the per-node and per-agent views of §5.5 are the details it hands off to.

Computing node health from raw events is $O(\text{events})$ per refresh, untenable as history grows. The system instead maintains *rollups*: for each (host, minute) pair, a bucket aggregating that minute's activity (event and error counts, latency sum/count/p95/max, token and cost totals, distinct agents). An emitter recomputes and appends full bucket snapshots to a dedicated rollup story; because each append is a complete snapshot of its cell, the reader keeps the last value seen per key and re-emission never double-counts: idempotence by last-writer-wins, the same discipline as §4.4. A health query then folds only the buckets in the requested window: $O(\text{nodes} \times \text{buckets})$, independent of event volume (R4). Pre-aggregating a monitoring stream into fixed time buckets at write time, rather than scanning raw points at read time, is the standard move in fleet-scale time-series monitoring [42]; the variant here differs in storing the buckets as ordinary events in the same log rather than in a separate database, so that the rollup inherits the durability and replayability of everything else and needs no second consistency story. Per-node health folds by explicit rules: sums for counts, a ratio for error rate, and the *maximum* of per-minute p95s as the window's p95 estimate (an upper-biased estimate that avoids retaining raw samples). Thresholds then classify each node ok/warn/critical. Nodes in the allocation that produced nothing are injected as empty rows, and every row is tagged with its live SLURM state, so the grid always shows the whole population. The emitter is optional. Absent rollups, the API falls back to folding raw events on demand, which is correct at small scale and keeps the tab from ever being empty.

5.4 Cross-Host Critical-Path Tracing

The third question, why this scenario is slow, is the classical tracing question of §2.2, posed across hosts. The tracer reconstructs call trees from stitched Path-B edges in two steps.

Tree reconstruction. Each stitched edge is a span: an interval on the timeline from `start` to `done`, with endpoints, tool, status, and latency. Parentage is resolved by a two-tier rule mirroring §4.3: if a span carries an explicit `parent_correlation_id`, that is its parent, the Dapper-style ground truth. Otherwise the parent is *inferred*: the span whose callee session matches this span's caller session and whose interval encloses this span's (the tightest such enclosure winning), on the Mystery-Machine premise that a delegation issued by agent B while B was servicing a call must be causally nested within it. Spans with no parent are roots; each root yields one trace, with depth assigned by tree walk.

Critical path and bottleneck. Within a trace, end-to-end latency is attributed by the analysis of §2.2.2 [54]: the critical path is walked from the root by descending into the slowest child at each level, and each hop on the path is charged its *self-time* (its latency minus its slowest child's), so that time spent waiting on a child is charged to the child, not double-counted in the parent. Reporting the slowest hop rather than an average is the point: at fleet scale the distribution's tail, not its centre, is what an operator experiences [15]. The hop with maximal self-time is named the *bottleneck*, with its hosts and tool: not "the scenario is slow" but "the `writer→reviewer` hop on `ares-comp-07`, 4.2 s of self-time." Because hosts ride on every span, the pivot to the fleet view is immediate: if the bottleneck's node also glows in the fleet heatmap, the cross-layer diagnosis (fleet symptom, physical cause) completes in two clicks. Chapter 6 follows exactly this pivot as a case study. The waterfall rendering shows the trace with depth indentation and the critical path highlighted; reconstruction is $O(s^2)$ in the spans of a scenario in the inference case (s small, tens of calls) and $O(s)$ with explicit parents.

5.5 The Operator Surface

Everything in this chapter and the last is, from the operator's chair, five tabs in one dashboard. Each view is a projection of the same captured event stream (none holds private state that is not reconstructible from the log), and each exists to answer one of the operator questions of §3.1. This section walks the surface as an operator sees it; Figures 5.1 through 5.5 are captures of the deployed system.

Scenarios: who is talking to whom (Fig. 5.1). The dashboard opens here. One scenario is selected from the left rail, its agents drawn as peers grouped into host containers, every inter-agent call an animated edge (color-coded by status, errors in red). The header carries the scenario's live totals (hosts, agents, messages, errors) and a *Live/Full* toggle: *Live* renders from the hot store and SSE bus as events arrive; *Full* re-reads the drained archive for complete fidelity, including a replay scrubber that re-animates history at chosen speed. Clicking any agent, host box, or edge opens its detail panel; from an agent the operator can pivot into its full conversation (the turns, token counts, and costs behind the node), which is the join of capture to conversation, exercised end-to-end by the functional harness of §6.6. Figure 5.2 shows the same view on a smaller scenario, where the edge styling is carrying the diagnosis: a delegation that started and never finished is rendered distinctly from one that failed, because the two demand different responses from the operator.

Health: what is wrong, and why (Fig. 5.3). The Doctor surface. The center column is the live LLM-call feed (every turn as it happens, filterable to errors or a provider); the left rail is the ranked incident list a scan produces: one card per clustered incident, ordered by severity, with occurrence counts and *recurring* badges read back from the incident history. Selecting an incident opens its diagnosis card: *why this is happening* and *how to fix it* in prose, the blast radius (occurrences, hosts, agents, scenarios), the heuristic's stated confidence, and the raw evidence samples that justify it. The compression argument of §5.2 is legible in the ratio: in Figure 5.3 the scan's 864 raw signals arrive as twelve ranked cards, each carrying its own justification, and the operator reads a dozen explanations rather than scrolling eight hundred errors.

Cluster: agent semantics on physical topology (Fig. 5.4). This view puts the storage layer itself on screen: the ChronoLog deployment drawn on the allocation's nodes (visor, one keeper card per node with the chronicles and stories it holds, grapher and player placement), overlaid with the cluster graph of §5.1 so that agent traffic is visible *on* the machines that carry it, next to the SLURM capacity badge (idle/busy/down). R7 reads directly off the picture: the same

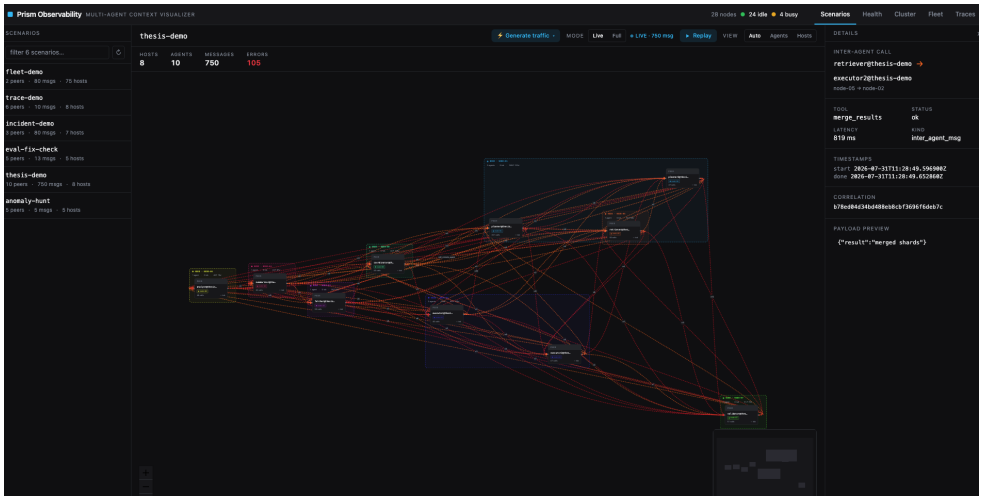


Figure 5.1: The Scenarios view, live. The `thesis-demo` scenario is selected from the left rail: 10 agents across 8 host groups, 750 messages and 105 errors on the live counters, every inter-agent call an edge (errors red). The right panel is a clicked edge: `retriever`→`executor2` across `node-05`→`node-02`, its tool, status, latency, correlation identifier and payload. Both endpoints resolve to their agents' conversations: the capture-to-conversation join of R1.

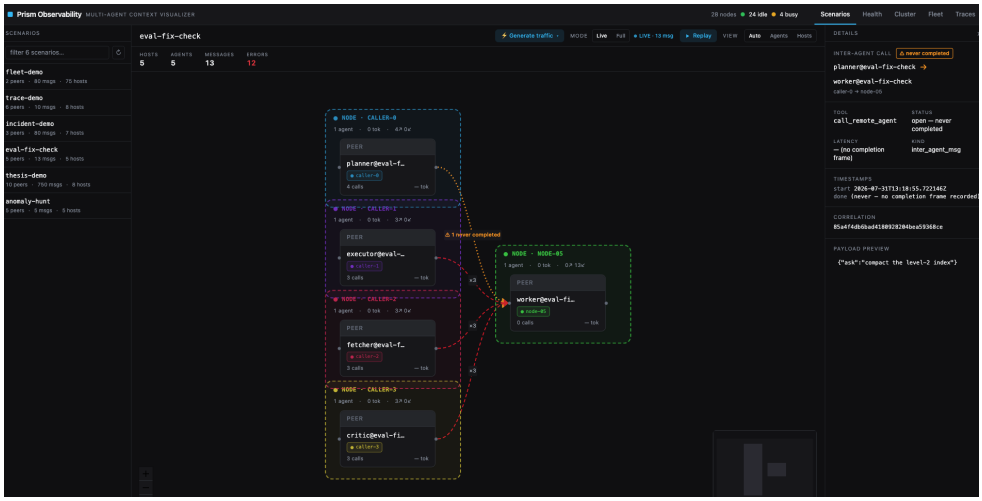


Figure 5.2: The same view on a smaller scenario, showing the failure whose signature is an *absence*. Four caller agents delegate to one `worker` on `node-05`; three delegations failed (red, dashed) and one *never completed at all*. That one is drawn amber and badged *1 never completed*, and its detail panel reports `open -- never completed` with no latency and no completion frame, only the correlation identifier of the `start`. This is the orphan call that §5.2 always ranks critical, and the rendering that §6.7 introduced after the case study showed open delegations were indistinguishable from completed ones.

physical node hosts both an agent's runtime and the keeper that captures it, and the operator sees both roles at once.

Fleet: the population at a glance. One cell per node, colored by rolled-up health state (ok / warn / crit, idle and busy dimmed), worst first; the header aggregates events, errors, and cost over the selected window. The representation change argued in §5.3 pays off here: where the node-link graph saturates at a dozen nodes, the heatmap stays legible at hundreds, and the red cells name the offending node before a single query is typed. Clicking a cell filters the fleet to that node and hands off to the other views for the why. The measured cost of this view, and the node counts at which it stays interactive, are reported in §6.5.

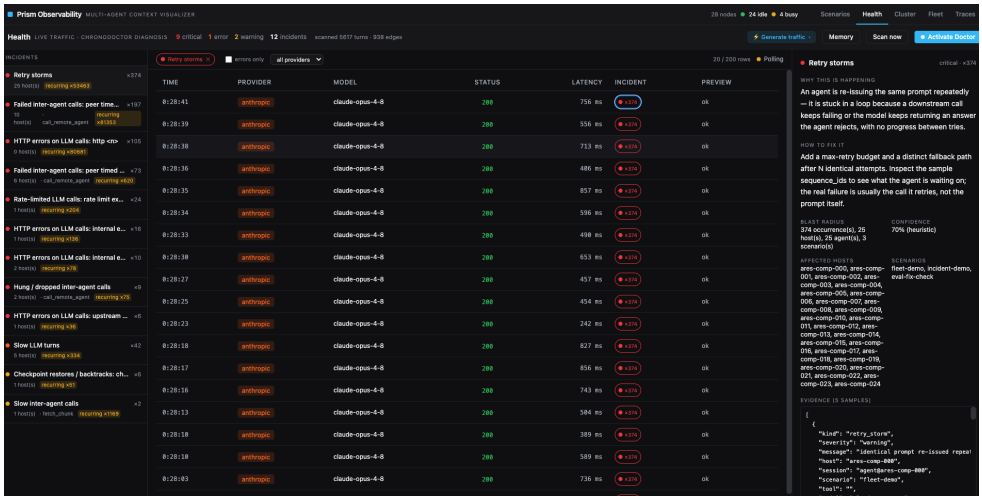


Figure 5.3: The Health view after a Doctor scan. The header reports the scan's extent and verdict: 5 617 turns and 938 edges gathered, condensed to **12 incidents** (9 critical, 1 error, 2 warning). The left rail ranks them worst-first, each with its occurrence count and a *recurring* badge read back from the cross-run incident history. The centre column is the live LLM-call feed, here filtered to the selected incident. The right panel is that incident's diagnosis: *retry storms*, 374 occurrences, with root cause and remediation in prose, a blast radius of 25 hosts, 25 agents and 3 scenarios, a stated confidence of 70% (heuristic), and the raw evidence samples that justify it.

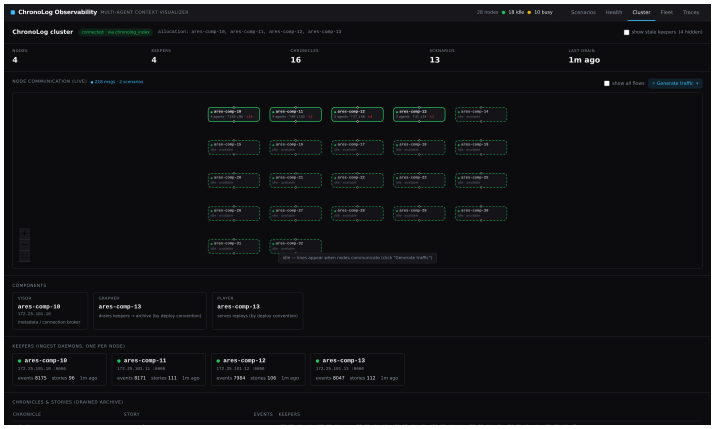


Figure 5.4: The Cluster view on a live 4-node deployment: ChronoLog components (visor, grapher, player placement; one keeper card per node with its event and story counts and last-drain age) alongside the live cluster graph and SLURM capacity. The headline freshness stat reports the *measured* last-drain age (§6.3); the grapher's configured 180 s acceptance window is relegated to its tooltip.

Traces: where the time went (Fig. 5.5). The critical-path surface of §5.4: traces listed slowest-first; the selected trace rendered as a depth-indented waterfall with the critical path outlined and each hop labeled *tool* → *host*; the header names the bottleneck hop with its self-time: in the capture, `call_remote_agent` → `comp-04` at 12.0 s of a 21.6 s trace. A self-time bar above the waterfall shows at a glance which host consumed the wall-clock, and clicking any span opens the edge's detail. The question of why the scenario is slow is answered by naming the responsible hop, without requiring the operator to read the trace.

Together the five views implement the pivot the case study of §6.7 times end-to-end: Fleet says *which machine*, Health says *what and why*, Traces says *where the time went*, Scenarios says *who*, and the conversation view says *show me*. All are projections of one shared log, reachable from each other in a click or two.

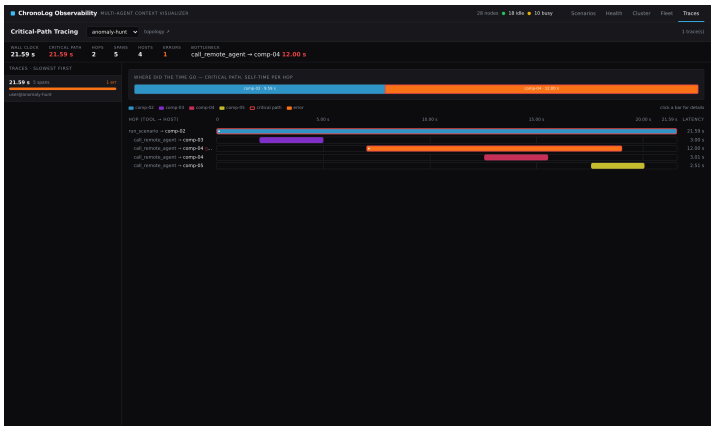


Figure 5.5: The Traces view: critical-path waterfall for the slowest trace of a scenario. The header names the bottleneck (the `comp-04` hop, 12.0 s self-time of 21.6 s wall-clock); the top bar shows self-time per hop along the critical path.

Chapter 6

Evaluation

The evaluation asks six questions, mapped to the requirements of §3.2: what does capture cost the observed workload (R5, §6.2); how fresh are the live views (R2, §6.3); is the Doctor's diagnosis *correct*, not merely cheap (R6, §6.4); do the analyses scale as claimed (R4, §6.5); does the whole stack work end-to-end on a real cluster with real agents (R1, R3, R7, §6.6); and does the system actually shorten the path from symptom to cause (§6.7). Section 6.8 discusses limitations. Table 6.1 previews the headline results.

6.1 Methodology

Platform. All live experiments run on the ARES cluster under SLURM. Each experiment allocates M idle compute nodes and deploys ChronoLog across them (visor, one keeper per node, grapher draining to an archive on shared storage), followed by the observability components: one collector and one capture-worker process per node, and one dashboard on a designated node. The deployments evaluated live use $M = 4$ and $M = 8$. In this configuration the keeper seals 10 s story chunks under a 15 s acceptance window, while the grapher and player run with a 180 s acceptance window; §6.3 measures what these settings translate to end-to-end, and corrects a folklore number in the process.

Workloads. Two workloads exercise the identical ingest path. *Real agents*: Claude Agent SDK agents, one or more per node (configurable as K agents per node), each performing genuine LLM turns and one cross-node delegation through a remote-agent MCP tool per round; turns enter Path-A, edges Path-B. *Synthetic traffic*: a generator that emits realistic interaction records and `start`/`done` edge pairs through the same spool and collector endpoints, with configurable topology (mesh, star, pipeline, ring), agent count, node spread, error rate, and pacing. Real agents provide ground truth end-to-end; synthetic traffic provides controlled, reproducible, zero-token-cost load for overhead and scalability measurements, and is the fault-injection vehicle for §6.7.

Measurements. Latency overhead is measured as the difference in per-turn wall-clock between instrumented and uninstrumented runs of the same agent workload; capture-side costs by sampling the sidecar processes' CPU and resident memory under load; storage by bytes per record in each tier; freshness by timestamping an event at emission and at receipt in a subscribed SSE client. Each is reported with distribution statistics over repeated runs rather than as a single number: the workload under test is a non-deterministic one running on shared cluster hardware, which is exactly the regime in which single-run comparisons are known to mislead, and in which a difference must be stated with a confidence interval or not stated at all [20]. Percentiles rather than means are reported wherever an operator's experience is governed by the tail [15].

6.2 Capture Overhead

Capture overhead is the cost the observed workload pays, and the design predicts it is small by construction: the Path-A fast path is one local JSONL append (no network, no lock, no downstream dependency, §4.4); the Path-B fast path is one HTTP POST to a daemon on `localhost` that enqueues and returns. Both are constant-time and independent of history size. Everything else (the ChronoLog write, the forward, the drain) happens in sidecar processes off the agent's critical path.

Question	Measured answer
What does capture cost the agent?	It adds 3.6 ms to a turn that averages 7.7 seconds, under 0.05%. Run against real agents with capture on and off, the difference was too small to measure (§6.2).
How fresh are the live views?	An event reaches the operator's screen in 10.7 ms. The durable archive takes 20–30 s to catch up, so the live view is roughly 1 900× faster (§6.3).
Is the diagnosis correct?	It found all 580 planted faults and raised no false alarms on clean data. Five hundred identical errors were presented as one ranked incident (§6.4).
Do the analyses scale?	Fleet queries stayed at 19–24 ms while stored events grew 50×, and grow linearly with fleet size: 49 ms at 400 nodes of synthetic state (§6.5).
Does it run for real?	The whole stack passed all 28 end-to-end checks on live 4- and 8-node clusters, running up to 32 real agents without per-node cost rising (§6.6, §6.5).
Does it shorten diagnosis?	An operator meeting the fault for the first time diagnosed it completely in 72 seconds. From the raw logs the same operator could not answer any of the three questions (§6.7).

Table 6.1: The evaluation at a glance. Every number is measured; the sections referenced give conditions, distributions, and caveats.

Figure 6.1 shows what capture adds to each event on the agent's critical path, and Table 6.2 the standing resource and storage costs that a latency plot cannot show. All latency numbers are taken on live compute nodes with the production NFS state directory: the real deployment condition, not a tmpfs microbenchmark.

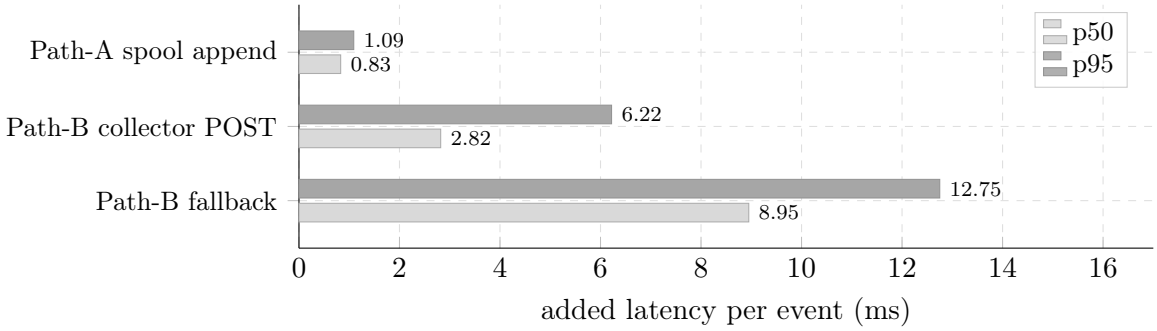


Figure 6.1: What capture adds to the agent's critical path, per event (n=10 000 per bar, live 8-node deployment, production NFS state directory). Path-A's spool append is a local file write and costs under a millisecond. Path-B's POST to the node-local collector costs a few. The bottom pair is the degraded mode taken when the collector is down: the agent posts directly to the dashboard, at roughly 3× the cost. Degradation is slower, not cheaper, which is why the collector exists. For scale, the mean LLM turn these sit inside is 7 734 ms, three orders of magnitude to the right of this axis.

The headline is easy to over- and under-claim, so it needs stating carefully. The A/B experiment ran n=30+30 real agent turns (identical prompts, interleaved to absorb provider drift): instrumented turns averaged 8 099 ± 5 147 ms, uninstrumented 7 734 ± 4 291 ms, a delta of +365 ms whose 95% confidence interval (±2 398 ms, Welch $t = 0.30$) makes it statistically indistinguishable from zero. The claim therefore rests on both measurements: the *mechanism* costs $0.83 + 2.82 \approx 3.6$ ms per turn-plus-delegation (under **0.05%** of a 7.7 s mean turn), and the A/B run confirms that nothing beyond the mechanism leaks onto the critical path. With the collector *down*, the fallback POST to the dashboard costs 3× the normal path (8.95 vs. 2.82 ms p50): degraded mode is slower, not cheaper, which is why the collector exists.

Two structural observations frame the numbers. First, the denominator is large: an LLM turn takes seconds and a delegation round-trip longer, so capture cost in milliseconds translates to a

Cost	Path	Metric	Measured
End-to-end turn delta	A+B	instrumented vs. not, per turn	$+365 \pm 2\,398$ **ms**
Collector daemon	B	CPU / RSS per node, 20 ev/s	32% / 40 MB
Capture worker	A	CPU / RSS per node, same load	0.8% / 51 MB
Interaction record	A	bytes per event (spool / archive)	2 093 / 2 173 B
Edge event	B	bytes per event (hot / archive)	612 / ≈560 B
Live forward	B	bytes per event on the wire	646 B

Table 6.2: The standing costs of capture, alongside the per-event latencies of Figure 6.1 (live 8-node deployment, compute nodes, NFS state directory). The first row is the end-to-end A/B turn delta over real LLM turns: its 95% confidence interval spans zero, so capture is statistically invisible inside the workload's own variance. Capture worker CPU is the mean of a bursty profile (idle between 5 s drain passes, peaks of 85% during them); the wire number is amortized over batches of ≤64 (658 B unbatched).

per-turn overhead orders of magnitude below the workload's own variance. This is the inverse of the classical tracing regime (§2.2.3), where sub-millisecond RPCs made even microseconds of instrumentation significant; it is why complete capture is affordable here. Second, the failure-mode cost is bounded by design rather than measured luck: if every downstream component dies, the agent's cost is unchanged (Path-A) or one failed local connection followed by a fallback POST (Path-B).

6.3 Freshness and Query Latency

Freshness is the interval from an agent emitting an event to that event being visible to an operator. The claim under test is that the hot path (R2) puts the event on screen in milliseconds while the cold path delivers durability in tens of seconds, and the measurement corrects a number this thesis itself repeated from deployment folklore.

Figure 6.2 plots the whole range on one axis: the hot-path pipeline (emit → collector → dashboard ingest → SSE frame at a subscribed client), each per-view query against the populated live deployment, and the cold-path drain they must all be read against. Four numbers do not fit on that axis and are given in Table 6.3: the tails, the degraded path, and the rollup-backed fleet read that §6.5 explains.

Measure		Measured
Emit → SSE frame at client	p95	11.8 ms
direct-ingest fallback	p50 / p95	8.5 / 8.9 ms
Cold-path availability (drain)	max, n=10	30 s
Fleet health, rollup-backed (§6.5)	p50	19–24 ms

Table 6.3: The freshness numbers Figure 6.2 cannot show: the tail of the live push, the degraded direct-ingest path taken when the collector is down, the worst drain observed, and the rollup-backed fleet read. Live 8-node deployment, dashboard node, n=100 per query row, n=1 000 paced events at 20 ev/s with zero loss for the SSE rows, corpus ≈22 k archived events.

The drain is 20–30 s; the "∼180 s" was something else. Measured directly (append a marked event, poll the archive), a story chunk becomes readable **20–30 s** after append (n=10, p50 20 s), governed by the keeper's 10 s chunking and 15 s acceptance window plus the grapher's write batch, not by the 180 s window configured downstream. The ∼180 s that operators (and earlier drafts of this thesis) attributed to the drain is in fact the client's *replay-query timeout*. In these deployments the player's playback responses never reach the dashboard process's receiver service: the player logs every request and completes none of the transfers toward that endpoint, while an otherwise-identical receiver in the capture process works. Every live replay issued by the dashboard therefore blocks for the full `CL_ERR_QUERY_TIMED_OUT` window of ∼180 s: per story,

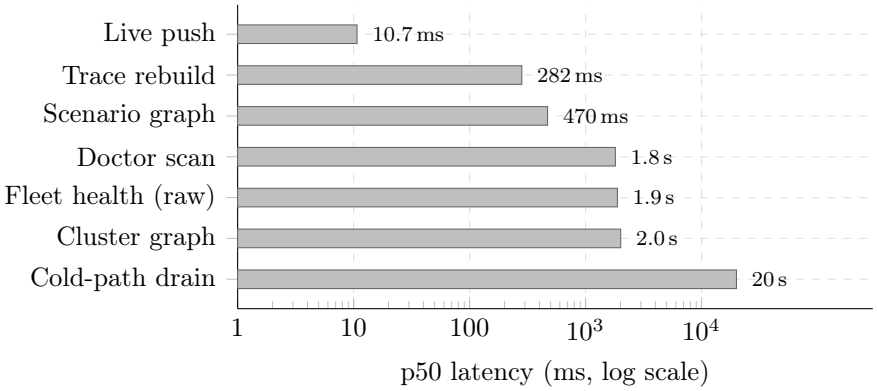


Figure 6.2: The freshness and query-latency spread on one log axis (p50, live 8-node deployment). Live push takes single-digit milliseconds and scenario-scoped reads stay sub-second, while population-wide reads that fold the raw archive sit near 2 s on this corpus. That is the linear-in-events cost the rollup design of §6.5 flattens to roughly 20 ms.

per request, holding the client lock. Diagnosing this (§6.8) explained the folklore number and motivated generalizing the archive-first read rule of §4.5 from listing indexes to *all* dashboard replays: with the drain at 20–30 s, serving full-fidelity views from the archive costs at most half a minute of staleness and removes the last blocking read from the serving process.

Qualitatively, the split then behaves as designed in every live run: the live views (scenarios, cluster, fleet, health, traces) render immediately during a run, while full-fidelity conversation views fill in within the half-minute drain; no view blocks the dashboard, fresh clusters included.

6.4 Detection Quality

R6 demands that diagnosis be free of marginal token cost; this section asks whether the deterministic pipeline that satisfies it is also *correct*. The experiment scores a full Doctor scan (gather, detect, cluster, diagnose, record; §5.2) against a ground-truth manifest of injected faults. Deliberately injecting the failure and checking that the system reports it is the discipline of controlled fault injection [9], applied here offline rather than in production so that the manifest is exact and the false-positive rate is measurable against a known-clean background. A controlled corpus is seeded offline through the same spool and inter-agent-store write paths the capture layer uses: a clean background of healthy traffic (216 turns, 120 completed delegations), plus ten planted faults of each of the eight detector families of §5.2 (5xx errors, rate-limiting, retry storms, latency outliers, edge errors, orphan calls, slow edges, recoveries), plus one *storm* (500 identical peer-timeout edges spread across 40 hosts) to test that section's compression claim. Each planted fault is shaped to carry exactly one symptom (single-label ground truth), so every detection is unambiguously attributable. The same fault mix is then injected a second time and the corpus re-scanned, testing whether the incident history recognizes the recurrence.

The result is a clean sweep: all 580 planted signals were detected, the eighty single faults and every event of the storm alike, and every incident was routed to its intended diagnosis rule, at confidences from 0.55 (latency outliers, slow edges) to 0.85 (rate limiting). The scan is seed-deterministic (`e6_doctor_quality.py` in `scripts/eval/`), and the result is identical at n=25 and under a different seed.

Three results carry the argument. First, the clean background produces *zero* incidents: the detectors' thresholds (medians with absolute floors, consecutive-repeat minima) do not fire on healthy variance, so a ranked incident list starts empty rather than requiring the operator to learn a noise floor. Second, compression behaves as designed: 580 injected raw signals become 9 ranked incident cards; in particular the 500 identical timeouts collapse into *one* card whose count, host list, and severity escalation carry the blast radius (the "not a scroll of per-event errors" the operator asked for). Third, the longitudinal claim holds mechanically: after the first scan writes

the incident memory, the second scan badges all nine incidents as recurring with the correct prior count, read back from the self-compacting memory document rather than from process state.

The scope of the claim is bounded. The planted faults are in-distribution and comfortably above the detector thresholds: the experiment establishes that detection, clustering, diagnosis routing, and cross-run memory are correct on unambiguous faults, not that the thresholds are well-calibrated on borderline cases. "Diagnosis correct" likewise means only that the incident reached the intended rule of the hand-built rule set, whose coverage limits are discussed in §6.8. A perfect score under these conditions is the expected behavior of deterministic detectors, not evidence against harder inputs; the informative results are the zero-false-positive background, the exact 500→1 compression, and the memory round-trip.

6.5 Scalability

The evaluation tests three scaling claims.

Fleet reads are independent of event volume. The rollup design (§5.3) makes the fleet query $O(\text{nodes} \times \text{buckets})$; the raw-fold fallback is $O(\text{events})$. The experiment holds nodes and window fixed (16 nodes, 60-minute window) and grows the event count from 10^4 to 5×10^5, comparing the fleet-health query latency of the two paths (20 requests per point, p50). Figure 6.3 shows the result: rollup-backed reads stay flat at 19–24 ms across a 50× growth in event volume, while the raw fold grows linearly (slope ≈17 ms per thousand events between the endpoints): 107 ms at 10 k events, 8.5 s at 500 k, a 457× gap at the largest size.

Fleet reads grow linearly in fleet size, and only in fleet size. The complementary experiment holds the per-node event density fixed and grows the *number of nodes*. It cannot be run on the live allocation: the largest cluster available here was eight nodes (§6.6). It is therefore run against *synthetic fleet state*: the harness invents n host names and writes their events through the production capture API, the production emitter bucketizes them, and the production dashboard is then queried over HTTP exactly as an operator would. What is synthetic is the origin of the events; the store, the query path and the measured latency are the real ones. This is sufficient for the claim under test, which is a property of the read algorithm: the fold cost depends on how many buckets exist, not on which machine produced them. It is correspondingly *not* evidence about capture, drain or network behaviour at that scale, which the live 4- and 8-node deployments cover instead.

Figure 6.4 shows the result over fleets of 10 to 400 nodes (50 requests per point). Latency is linear in node count, $5.2 + 0.105n$ ms with $R^2 = 0.98$: a fixed ∼5 ms of request handling plus ∼0.1 ms per node summarized. A 100-node fleet is answered in 14.7 ms p50 and a 400-node fleet in 49.0 ms, both inside interactive time, where the same query folded from raw events would touch every record ever captured. The 100-node point reproduces an independent measurement taken on the cluster three weeks earlier (15.1 ms), which is the reason to trust the rest of the curve. The sweep stops at 400 nodes because that is the emitter's configured session bound (`max_sessions`, §5.3); beyond it the harness silently re-measures a 400-node fleet rather than a larger one.

Representation, not just computation, must scale. The node-link cluster graph is legible to roughly a dozen nodes and degenerates beyond; this is a limit of the visual form, not of the data. The fleet heatmap holds one cell per node and remains legible at hundreds; its rendering cost grows with nodes only. This is why the system changes representation between the topology view (small M, maximal detail) and the population view (large M, one health value per node).

Agents per node. The real-agent harness scales in two dimensions: nodes (M) and agents per node (K). Capture is per-node (one collector, one keeper, one worker regardless of K), so

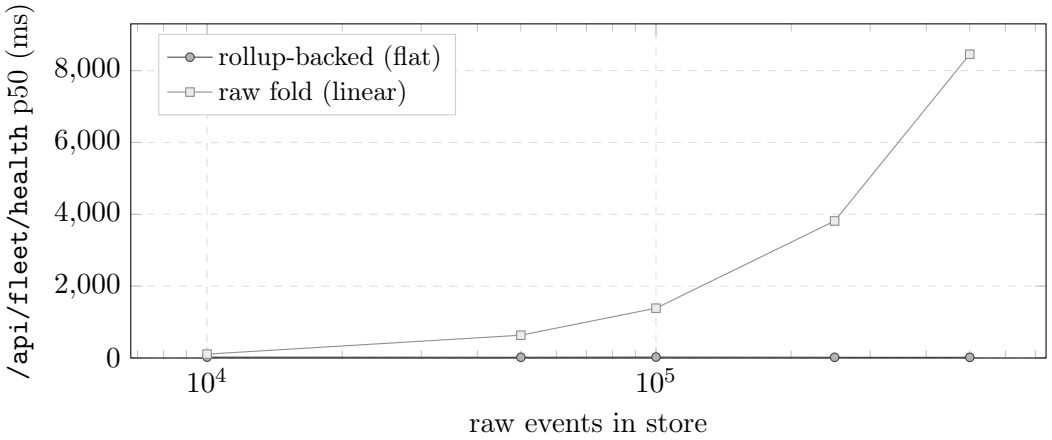


Figure 6.3: Fleet-health query latency vs. stored event count (16 nodes, 60-minute window, p50 over 20 requests). Pre-aggregated rollup reads are independent of event volume; the raw-fold fallback is linear in it. Measured offline via `bench_rollup_scaling.py` in `scripts/eval/`.

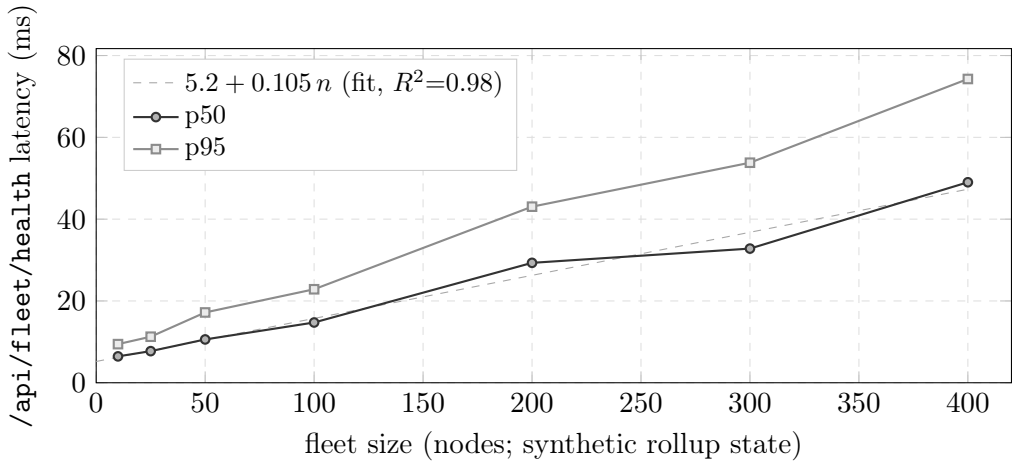


Figure 6.4: Fleet-health query latency vs. *fleet size* (50 requests per point), the complement to Figure 6.3: there the node count is fixed and events grow, here events per node are fixed and the fleet grows. Latency is linear in nodes, $5.2 + 0.105n$ ms ($R^2 = 0.98$). **The fleet state is synthetic**: the harness invents n host names and writes their events through the production capture path, so the store, the query and the timings are real but no n-node cluster is involved. The largest *live* deployment in this thesis is eight nodes (§6.6). The sweep stops at 400 nodes, the emitter's configured session bound. Measured on the ARES gateway via `bench_rollup_scaling.py` in `scripts/eval/`.

per-node capture load grows linearly in K with no cross-node coordination, and cross-node load grows only with actual inter-agent traffic. Live runs swept $K \in \{1, 2, 4\}$ at $M = 8$ with real agents (every agent performing genuine LLM turns and one cross-node delegation): all 8, 16, and 32 agents exited cleanly and every delegation round-tripped through the archive (16, 32, and 64 edge records respectively). The per-node sidecar was flat across the sweep (Figure 6.5): the collector held 0.1–0.3% mean CPU (transient forward bursts to ∼21%) and 37–39 MB RSS at every K, and the capture worker 0.3% mean at 48 MB. Neither series is monotonic in K: CPU falls from K=1 to K=2 and rises again at K=4, which is the signature of measurement noise around a constant rather than of a trend, and is the result the design predicts: one collector, one keeper and one worker serve a node whatever K is. These means sit far below Table 6.2's 32% figure because the regimes differ: the table's collector serves a sustained 20 ev/s synthetic stress load, while real agents (one turn every several seconds) offer a small fraction of that rate. Both observations are consistent with the claim that agent count stresses the LLM provider, not the capture plane. The largest configuration exercised cleanly end-to-end is therefore $K \times M = 4 \times 8 = 32$ real agents.

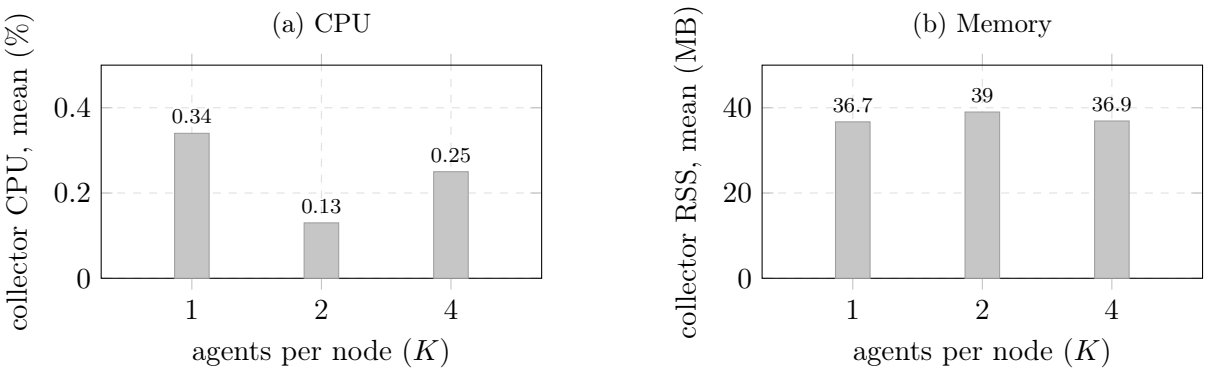


Figure 6.5: Per-node capture load as agents per node quadruples, measured on the live 8-node sweep with real agents ($K \in \{1, 2, 4\}$, i.e. 8, 16 and 32 agents in total). Both axes start at zero, and neither quantity trends: the collector's mean CPU is non-monotonic in K (noise around a constant, not growth) and its resident memory moves by ∼2 MB across the sweep. This is the design's prediction (one collector, one keeper and one capture worker serve a node regardless of how many agents run on it), so adding agents costs the LLM provider, not the capture plane. CPU and memory are drawn on separate axes because they share no unit; the transient forward bursts to ∼21% are not shown, being spikes rather than sustained load.

6.6 Functional Validation

The system's end-to-end correctness claim, capture through the log store to every view with real agents on a real cluster (R1, R3, R7), is validated by an automated harness rather than by demonstration. The harness brings up the full deployment on a live allocation, runs real Claude agents (genuine LLM turns; one cross-node MCP delegation per agent), waits out the drain, restarts the dashboard cold against the same log store (so everything shown is reconstructed from durable state, not process memory), and then asserts machine-checkable properties of every API the dashboard serves: each view is populated with the expected entities, per-agent metrics are non-zero where turns occurred, and inter-agent edges exist with correct endpoints. The strictest check is the cross-plane join: the scenario graph's clicked node resolves to that agent's conversation with its turns and token counts, exercising the session-identity contract of §4.3 end to end.

The harness passes **28/28** checks on live 4-node and 8-node ChronoLog clusters. Beyond the pass count, the harness is the regression suite the design was corrected against: the read-only rule, the archive-first index read, and the hostname canonicalization of §4.6 were each introduced to turn a live-cluster failure of this harness into a pass, and remain pinned by it.

6.7 Case Study: Fault Injection and Localization

The final question is whether the pieces compose into the cross-layer diagnosis the thesis promises. The scenario: on a live M-node deployment carrying steady background traffic, the synthetic generator injects a retry/timeout storm concentrated on one node, alongside a delegation leg that starts and never completes. The fault is deliberately of the fail-slow rather than fail-stop kind [21]: the node stays up, keeps answering, and merely degrades, which is the case a liveness check cannot see and a population comparison can.

The observed localization path runs one view per operator question. A Doctor scan returns not hundreds of error rows but a short ranked list: one incident aggregating the storm (count in the hundreds, blast radius naming the affected hosts and sessions, root cause and remediation attached) and one *critical* orphan-call incident for the hung delegation, invisible in any per-edge view because its signature is an absence. The fleet heatmap turns "the fleet looks unhealthy" into "it is this machine": one red cell among ok nodes. The trace waterfall for the affected scenario charges the bottleneck hop by self-time, landing on that same node. Clicking the offending agent opens its conversation, the actual turns, retries and costs behind the symptom, completing the

pivot in two clicks. Re-running the fault and re-scanning badges the incident `recurring` $\times N$: the second occurrence is recognized, not rediscovered. Between them the five operator questions of §3.1 are answered in one pass over one fault.

Timed comparison. The path above was then measured as a controlled exercise on the live 8-node deployment. Faults were injected by script (a 6-turn identical-prompt 5xx retry storm on one node, twelve peer-timeout delegations into it, and one delegation leg killed after its `start` frame), and a diagnostician answered three questions, once through the dashboard and once from the raw capture files with shell tools only, against a verifiable ground truth: *which node is failing, and with what error*; *which session is retrying, and on what prompt*; and *which delegation never completed*, named by caller→callee and correlation id. The operator was the author, timing each question from first interaction to stated answer, with every answer checked against the injected values (including the orphan's correlation id). Table 6.4 reports the result.

Operator	Interface	Node	Session	Orphan	Outcome
Human, first exposure	dashboard	35 s	17 s	20 s	3/3 correct (72 s)
Human, second pass	dashboard	10 s	5 s	5 s	3/3 correct (20 s)
Human, third pass	dashboard	5 s	5 s	5 s	3/3 correct (15 s)
Human	raw logs	abandoned at ∼5–10 min			**0/3**
LLM agent	raw logs	44 s wall-clock, 4 commands			3/3 correct

Table 6.4: Timed diagnosis of the injected fault, n=1 operator (first-ever exposure to the task). The three timed columns are the three questions above, in the order asked. Dashboard trials repeat the same questions, so passes 2–3 measure a practiced operator. The raw-log attempt ran *after* the dashboard trials (answer leakage, if any, favored the raw-log path) and still produced no answers: the operator could not establish a starting point in the JSONL directories. The supplementary row is a fresh LLM agent given only the three questions and the data directories (no ground truth, shell text tools only); machine wall-clock is not comparable to human minutes and is reported for what it demonstrates about the *data*, not the operator.

Two findings, one expected and one not (the second previewed in §2.6.3). The expected one: a first-time operator produced a complete, verified diagnosis through the dashboard in **72 seconds**, yet could not answer *any* of the three questions from the raw files, abandoning after five to ten minutes without a foothold. The dashboard run included recovering from one wrong pick on the orphan question: the twelve errored-but-completed delegations rendered adjacent to the true orphan, and the correlation-id check caught the confusion. The unexpected one: the LLM-agent baseline answered all three questions from the raw JSONL in 44 seconds and four shell commands, finding the orphan by counting correlation-id occurrences, the very question the protocol predicted would time out under `grep`. Together they sharpen what this chapter can claim: the capture layer's raw data is *sufficient* for complete diagnosis, but it is not *accessible*: extracting the answer requires either expertise the on-call operator does not have or tooling that encodes it. The dashboard is that encoding: it substitutes for operator expertise, not for data.

The caveats are plain: one operator, one injected incident on a small scenario corpus, and a UI defect surfaced by the stumble on the orphan question (never-completed delegations rendered indistinguishably from completed ones). That defect was fixed following the study. Open delegations now carry a dedicated visual class (amber, dashed, badged *never completed*) across the scenario graph, the edge detail panel, and the trace waterfall, and hung flows outrank errors in edge selection; the pathology that cost the operator their one wrong pick is now the most visually distinct element on screen (Figure 5.2).

6.8 Discussion and Limitations

Hot store locality. The hot store and live bus live with the dashboard process; they are not replicated. A dashboard failure loses no data (the log is authoritative; the hot view rebuilds from

backlog and, after the drain, from ChronoLog) but interrupts live observation: acceptable for an operator tool, but a real asymmetry with the replicated cold path.

Two properties of the deployment, not of this design. The measured 20–30 s cold-path latency follows from the ChronoLog deployment's chunking and acceptance-window configuration rather than from anything this system does; the design works around it rather than removing it, and a deployment with tighter chunking would narrow the hot/cold gap without changing the architecture.

A second property surfaced during the evaluation itself. In the deployments measured here, playback responses from the player were not observed to reach the dashboard process's receiver service: the requests appear in the player's log, no transfer toward that endpoint completed during these runs, and an otherwise-identical receiver hosted in the capture process did receive them. The effect is that a live replay issued from the dashboard ran to the ∼180 s query timeout while holding the client lock. This thesis does not diagnose the cause and does not claim one; the observation is scoped to the version, configuration and client binding deployed here, and is reported so that the workaround it motivated is reproducible rather than unexplained. That workaround was to generalize the archive-first rule: with `CHRONOLOG_ARCHIVE_FIRST` set, the dashboard and capture worker serve all replay reads from the grapher's archive and never ask the player, bounding worst-case staleness at the measured drain window. It is the right trade under the behavior observed, but it leaves the live-replay path unused, and a deployment in which that path returns would recover it.

Diagnosis coverage. The heuristic diagnoser is first-match-wins over a hand-built rule set: specific on the failure classes it encodes, and explicitly low-confidence outside them. It explains known pathologies; it does not discover novel ones. The signature scheme can also split one logical failure into two incidents if its message text varies beyond what normalization strips.

Evaluation scale. Live validation ran at $M \in \{4, 8\}$ with real agents (the allocation sizes available), while the 100+-node claims rest on the rollup complexity argument and seeded fleet-scale data rather than a live fleet of that size. The p95 fold of §5.3 is upper-biased by construction, and inferred (parentless) traces rely on time nesting, which can mis-parent overlapping concurrent delegations between the same pair of agents; explicit parent propagation removes the ambiguity where the instrumentation provides it.

Taken together, the six questions this chapter opened with have quantitative answers: capture is statistically invisible to the agent, live views run three orders of magnitude ahead of the durable drain, detection is exact on ground truth, the fleet reads scale as designed, the stack survives real agents on real clusters, and the path from symptom to cause takes an untrained operator seventy-two seconds. What these answers add up to, and what they cost, is the subject of the concluding chapter.

Chapter 7

Conclusions and Future Work

This chapter closes the thesis: it checks the outcome against the requirements of Chapter 3, states the design lessons the work argues for, and lays out the directions it opens.

7.1 Conclusions

This thesis set out from a one-sentence gap: the tools we surveyed answer "what did my agent do?" with a trace of one run on one machine, and, to our knowledge, none answers "what is my agent *fleet* doing across the cluster, and which node is dragging it down?" The hypothesis defended across the preceding chapters is that the gap closes naturally once the data is stored in its native shape: a multi-agent execution is a causally-ordered event log, and a distributed shared log with a capture process on every node makes the coupling of agent semantics to physical topology a property of the substrate rather than an integration problem. Section 3.3 states the contract a substrate must honour, which makes the hypothesis checkable against any candidate store, and Chapter 4 is written against it, with §4.8 isolating what implementing it on ChronoLog in particular required.

All seven requirements of §3.2 are met at the scale evaluated, each with a measured answer from Chapter 6. **R1** is met by the two capture paths and the `role@scenario` identity contract, under which every event carries its host and the planes join by construction. **R2** and **R3** are met jointly by the hot/cold split: an event reaches the operator's screen 10.7 ms after emission and the durable archive 20–30 s after, with at-least-once spooled capture deduplicated by high-water marks rebuilt from the log itself. **R4** required changing both computation and representation: per-(host, minute) rollups hold the fleet read at 19–24 ms across a 50× growth in event volume where the raw fold grows 457× slower, and the heatmap answers a 400-node fleet in 49 ms where node-link rendering fails at a dozen. **R5** holds structurally and empirically: 3.6 ms per turn-plus-delegation, under 0.05% of a mean turn, with an A/B delta statistically indistinguishable from zero. **R6** rests on the deterministic detect–cluster–diagnose pipeline, LLM assistance strictly opt-in, scoring 580/580 planted faults with zero false positives and compressing a 500-signal storm to one ranked card. **R7** is demonstrated at that scale by the live harness: 28/28 automated checks on 4- and 8-node clusters without modifying ChronoLog, the scheduler, or the agent SDK, sustained at 32 concurrent real agents with flat per-node sidecar load. Whether the same holds on an allocation an order of magnitude larger is untested.

Beyond the artifact, the thesis argues two transferable design lessons. The first is architectural. Observability systems are conventionally assembled as instrumentation plus a bespoke backend; for workloads whose events are few, expensive, and causally structured (as LLM-agent fleets are), inverting the architecture around a general distributed shared log yields the cross-layer joins, the durability, and the replayability for free, at the price of engineering around the log's read behavior. That price is real but modest, and the evaluation quantified both of its parts: the planned part (the hot/cold split of Chapter 4) and the unplanned part (the replay behavior reported in §6.8, absorbed by generalizing archive-first reads at a bounded cost of half a minute of staleness). In return it provides the capability neither surveyed camp offers: the pivot from a fleet-wide symptom to a single agent's conversation on a named machine. The case study measured that pivot at 72 seconds, questions to verified answers, for an operator who had never seen the task before.

The second lesson comes from the same case study's baseline, and it reframes what an observability *interface* is for. The raw captured log proved *sufficient* for complete diagnosis (an LLM agent given only the data directories answered all three questions, orphaned delegation

included, in under a minute), while the human operator, given the same files, could not begin. What the dashboard contributes is therefore the expertise rather than the access: each view is a frozen form of the queries an expert would write. Systems built on a sufficient substrate can choose where that expertise lives (in views for humans, or in agents that query the log directly), and the shared-log architecture serves both consumers from the same store.

7.2 Future Work

A second implementation of the contract. The work that would most directly test this thesis is a port to another store. The specification side is done: §3.3 states which features a store must provide and which it may omit at a priced cost, §4.7 turns that into the adapter interface, and §4.8 predicts feature by feature what a port would add and delete. What the contract has not had is an implementation that could falsify it. Writing two or three (a partitioned commit log such as Kafka, an embedded analytical store over the archive tiers, a local JSONL store for single-node development) and checking the predictions of Table 3.4 against what each port actually costs would settle it. A prediction that survives makes the contract a contribution; one that does not shows where it is under-specified, which is the more useful outcome.

Upstream convergence. Two of the three constraints this design works around are being removed at the source: in ongoing collaboration with the ChronoLog developers, the blocking replay of C2 and the missing catalog operation of C3 have since been addressed upstream. Their removal would retire real machinery here, making the read-only process split a preference rather than a necessity and the `__index__` chronicle redundant for listing. C1, the acceptance window, is inherent to the ingest-optimized design point and will remain, so the hot/cold split stays; that is the correct outcome, since it is the part of the read design that is about the class of substrate rather than one implementation of it. Re-measuring §6.3 against the fixed client would show how much of Chapter 4 was design and how much was workaround.

Smaller extensions. Four follow-ons sit within reach of the current implementation: ad-hoc queries evaluated along causal edges, the Pivot-Tracing move [34], which the log's ordering and correlation schema already make expressible; promoting the hot store and live bus to a small replicated service, so that more than one dashboard can watch a fleet; learning the diagnoser's rule set from the incident history rather than hand-building it; and exposing the log and its analyses as an MCP tool, so that a supervising agent could ask the questions the case-study operator asked. Live validation at hundreds of real nodes, the gap §6.8 concedes, is chiefly an allocation problem.

The direction of the field is not in doubt: agent systems are growing in both population and physical extent, and the questions operators ask are increasingly the questions distributed systems have always asked, now with a conversation attached. This thesis's contribution is to show that the sixty-year-old toolbox of that discipline, from happened-before to the shared log, is the one this workload needs.

Bibliography

[1] Anthony Agelastos, Benjamin Allan, Jim Brandt, Paul Cassella, Jeremy Enos, Joshi Fullop, Ann Gentile, Steve Monk, Nichamon Naksinehaboon, Jeff Ogden, Mahesh Rajan, Michael Showerman, Joel Stevenson, Narate Taerat, and Tom Tucker. The lightweight distributed metric service: A scalable infrastructure for continuous monitoring of large scale computing systems and applications. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 154–165, 2014.

[2] AgentOps. Agentops: Observability for AI agents. `https://www.agentops.ai`, 2026. Accessed July 2026.

[3] Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024. Specification; accessed July 2026.

[4] Apache Software Foundation. Apache pulsar: Cloud-native, distributed messaging and streaming. `https://pulsar.apache.org`, 2026. Documentation; accessed August 2026.

[5] Arize AI. Phoenix: Open-source AI observability and evaluation. `https://phoenix.arize.com`, 2026. Accessed July 2026.

[6] Mahesh Balakrishnan, Jason Flinn, Chen Shen, Mihir Dharamshi, Ahmed Jafri, Xiao Shi, Santosh Ghosh, Hazem Hassan, Aaryaman Sagar, Rhed Shi, Jingming Liu, Filip Gruszczynski, Xianan Zhang, Huy Hoang, Ahmed Yossef, Francois Richard, and Yee Jiun Song. Virtual consensus in Delos. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 617–632, 2020.

[7] Mahesh Balakrishnan, Dahlia Malkhi, Vijayan Prabhakaran, Ted Wobber, Michael Wei, and John D. Davis. CORFU: A shared log design for flash clusters. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 1–14, 2012.

[8] Paul Barham, Austin Donnelly, Rebecca Isaacs, and Richard Mortier. Using Magpie for request extraction and workload modelling. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI 04)*, pages 91–110, 2004.

[9] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016.

[10] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*, pages 221–227, 2021.

[11] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? arXiv preprint arXiv:2503.13657, 2025.

[12] Mike Y. Chen, Emre Kıcıman, Eugene Fratkin, Armando Fox, and Eric Brewer. Pinpoint: Problem determination in large, dynamic internet services. In *Proceedings of the International Conference on Dependable Systems and Networks (DSN 02)*, pages 595–604, 2002.

[13] Michael Chow, David Meisner, Jason Flinn, Daniel Peek, and Thomas F. Wenisch. The mystery machine: End-to-end performance analysis of large-scale internet services. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 217–231, 2014.

[14] Datadog. LLM observability. `https://www.datadoghq.com/product/llm-observability/`, 2026. Accessed July 2026.

[15] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[16] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS 17)*, pages 1285–1298, 2017.

[17] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. The many faces of publish/subscribe. *ACM Computing Surveys*, 35(2):114–131, 2003.

[18] Colin J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference (ACSC)*, pages 56–66, 1988.

[19] Rodrigo Fonseca, George Porter, Randy H. Katz, Scott Shenker, and Ion Stoica. X-Trace: A pervasive network tracing framework. In *Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation (NSDI 07)*, pages 271–284, 2007.

[20] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA 07)*, pages 57–76, 2007.

[21] Haryadi S. Gunawi, Riza O. Suminto, Russell Sears, Casey Golliher, Swaminathan Sundararaman, Xing Lin, Tim Emami, Weiguang Sheng, Nematollah Bidokhti, Caitie McCaffrey, et al. Fail-slow at scale: Evidence of hardware performance faults in large production systems. In *16th USENIX Conference on File and Storage Technologies (FAST 18)*, pages 1–14, 2018.

[22] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. Drain: An online log parsing approach with fixed depth tree. In *IEEE International Conference on Web Services (ICWS)*, pages 33–40, 2017.

[23] Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. Experience report: System log analysis for anomaly detection. In *IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, pages 207–218, 2016.

[24] IEEE and The Open Group. IEEE std 1003.1-2017, standard for information technology—portable operating system interface (POSIX). The Open Group Base Specifications Issue 7, 2018 edition, 2018.

[25] Flavio P. Junqueira, Ivan Kelly, and Benjamin Reed. Durability with BookKeeper. *ACM SIGOPS Operating Systems Review*, 47(1):9–15, 2013.

[26] Jonathan Kaldor, Jonathan Mace, Michał Bejda, Edison Gao, Wiktor Kuropatwa, Joe O'Neill, Kian Win Ong, Bill Schaller, Pingjia Shan, Brendan Viscomi, Vinod Venkataraman, Kaushik Veeraraghavan, and Yee Jiun Song. Canopy: An end-to-end performance tracing and analysis system. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP 17)*, pages 34–50, 2017.

[27] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.

[28] Anthony Kougkas, Hariharan Devarajan, Keith Bateman, Jaime Cernuda, Neeraj Rajesh, and Xian-He Sun. ChronoLog: A distributed shared tiered log store with time-based data ordering. In *Proceedings of the 36th International Conference on Massive Storage Systems and Technology (MSST 20)*, 2020.

[29] Jay Kreps. The log: What every software engineer should know about real-time data's unifying abstraction. LinkedIn Engineering Blog, `https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying`, 2013.

[30] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.

[31] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[32] LangChain, Inc. Langsmith: Observability and evaluation for LLM applications. `https://smith.langchain.com`, 2026. Accessed July 2026.

[33] Langfuse. Langfuse: Open-source LLM engineering platform. `https://langfuse.com`, 2026. Accessed July 2026.

[34] Jonathan Mace, Ryan Roelke, and Rodrigo Fonseca. Pivot tracing: Dynamic causal monitoring for distributed systems. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP 15)*, pages 378–393, 2015.

[35] Nathan Marz and James Warren. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.

[36] Friedemann Mattern. Virtual time and global states of distributed systems. In *Parallel and Distributed Algorithms*, pages 215–226. North-Holland, 1989.

[37] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[38] Neha Narkhede, Gwen Shapira, and Todd Palino. *Kafka: The Definitive Guide*. O'Reilly Media, 2017.

[39] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.

[40] Adam Oliner, Archana Ganapathi, and Wei Xu. Advances and challenges in log analysis. *Communications of the ACM*, 55(2):55–61, 2012.

[41] OpenTelemetry. Semantic conventions for generative AI systems. `https://opentelemetry.io/docs/specs/semconv/gen-ai/`, 2026. Accessed July 2026.

[42] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. Gorilla: A fast, scalable, in-memory time series database. *Proceedings of the VLDB Endowment*, 8(12):1816–1827, 2015.

[43] Thang Duc Pham, Harikrishna Tummalapalli, Fakhrul Hasan Bhuiyan, Álvaro Vázquez Mayagoitia, Christine Simpson, Riccardo Balin, Venkatram Vishwanath, and Murat Keçeli. Multi-agent orchestration for high-throughput materials screening on a leadership-class system. arXiv preprint arXiv:2604.07681, 2026.

[44] Pixie Authors. Pixie: Instant kubernetes-native application observability. `https://px.dev`, 2026. CNCF sandbox project; accessed July 2026.

[45] Prometheus Authors. Prometheus: Monitoring system and time series database. `https://prometheus.io`, 2026. CNCF graduated project; accessed July 2026.

[46] Redis Ltd. Redis streams. `https://redis.io/docs/latest/develop/data-types/streams/`, 2026. Documentation; accessed August 2026.

[47] Mendel Rosenblum and John K. Ousterhout. The design and implementation of a log-structured file system. *ACM Transactions on Computer Systems*, 10(1):26–52, 1992.

[48] Ben Shneiderman. The eyes have it: A task by data type taxonomy for information visualizations. In *Proceedings of the IEEE Symposium on Visual Languages*, pages 336–343, 1996.

[49] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical Report dapper-2010-1, Google, Inc., 2010.

[50] Renan Souza, Timothy Poteet, Brian Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. LLM agents for interactive workflow provenance: Reference architecture and evaluation methodology. In *SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (WORKS)*, 2025.

[51] WHATWG. HTML living standard, section 9.2: Server-sent events. `https://html.spec.whatwg.org/multipage/server-sent-events.html`, 2026. Accessed August 2026.

[52] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *First Conference on Language Modeling (COLM)*, 2024.

[53] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan. Detecting large-scale system problems by mining console logs. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP 09)*, pages 117–132, 2009.

[54] Cui-Qing Yang and Barton P. Miller. Critical path analysis for the execution of parallel and distributed programs. In *Proceedings of the 8th International Conference on Distributed Computing Systems (ICDCS)*, pages 366–373, 1988.

[55] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[56] Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing (JSSPP)*, volume 2862 of *Lecture Notes in Computer Science*, pages 44–60. Springer, 2003.